

РЕФЕРАТ

Научно-исследовательская работа состоит из 91 страниц, 22 рисунков, 6 таблиц, 28 использованных источников, 3 приложений.

**АСИМПТОТИЧЕСКИЙ АНАЛИЗ, РЕКУРРЕНТНЫЕ
СООТНОШЕНИЯ, СОРТИРОВКА БЭТЧЕРА, ЧЕТНО-НЕЧЕТНОЕ
СЛИЯНИЕ, СОРТИРОВОЧНАЯ СЕТЬ, КОМПАРАТОР, ГЛУБИНА СЕТИ,
СЛОЖНОСТЬ АЛГОРИТМОВ, МЕТОД РАСКРУТКИ,
ТЕЛЕСКОПИЧЕСКОЕ СУММИРОВАНИЕ, КЛАССИЧЕСКИЕ
АЛГОРИТМЫ СОРТИРОВКИ.**

Объект исследования — алгоритмы сортировки, прежде всего четно-нечетная сортировка слиянием Бэтчера и классические методы сортировки.

Предмет исследования — методы асимптотического анализа сложности алгоритмов, а также границы применимости данных методов.

Цель работы — проведение асимптотического анализа алгоритма сортировки Бэтчера, получение точных формул числа компараторов и глубины сортировочной сети, а также сравнение полученных результатов с классическими методами сортировки.

В работе рассмотрены теоретические основы асимптотического анализа, классификация алгоритмов сортировки. Получены точные формулы для числа компараторов операции четно-нечетного слияния, числа компараторов полной сортировочной сети Бэтчера и глубины сети. Выполнено сравнение сортировки Бэтчера с сортировкой слиянием, быстрой сортировкой, битонической сортировкой, сортировкой вставками и пирамидальной сортировкой. Разработаны программные реализации алгоритмов сортировки со счетчиками сравнений, проведен машинный эксперимент и подтверждено соответствие эмпирических результатов теоретическим оценкам.

СОДЕРЖАНИЕ

СПИСОК ТЕРМИНОВ И СОКРАЩЕНИЙ	5
ВВЕДЕНИЕ	9
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АСИМПТОТИЧЕСКОГО АНАЛИЗА АЛГОРИТМОВ СОРТИРОВКИ	17
1.1 Обзор предметной области	17
1.1.1 Алгоритмы сортировки и их классификация.....	17
1.1.2 Сортировочные сети: определение и свойства	20
1.1.3 Чётно-нечётная сортировка слиянием Бэтчера	22
1.2 Методы асимптотического анализа алгоритмов	25
1.2.1 Асимптотические обозначения.....	26
1.2.2 Метод решения рекуррентных соотношений.....	27
1.2.3 Мастер-теорема и метод Акра–Баззи	29
1.2.4 Метод подстановки и метод дерева рекурсии.....	30
1.2.5 Прямой подсчёт, вероятностный и амортизированный анализ	32
1.3 Информационно-теоретическая нижняя граница сортировки сравнениями	33
1.4 Постановка задачи исследования	36
Выводы по первой главе.....	37
2 МАТЕМАТИЧЕСКИЙ АНАЛИЗ СЛОЖНОСТИ АЛГОРИТМОВ СОРТИРОВКИ МЕТОДОМ РЕКУРРЕНТНЫХ СООТНОШЕНИЙ.....	40
2.1 Доказательство асимптотической сложности сортировки Бэтчера	40
2.1.1 Сложность операции слияния (Odd-Even Merge).....	41
2.1.2 Сложность полной сортировки.....	43
2.1.3 Глубина сети и параллельная сложность	45
2.2 Применение метода к классическим алгоритмам сортировки	47
2.2.1 Сортировка слиянием (Merge Sort)	47
2.2.2 Быстрая сортировка (Quick Sort)	48

2.2.3 Битоническая сортировка (Bitonic Sort)	51
2.2.4 Сортировка вставками и пирамидальная сортировка	52
2.3 Границы применимости метода рекуррентных соотношений	54
2.3.1 Необходимые условия применимости	54
2.3.2 Алгоритмы, требующие альтернативных методов анализа	55
2.4 Применение метода к криптографическим алгоритмам	57
2.4.1 Умножение Карацубы и алгоритм Тоома–Кука	57
2.4.2 Быстрое преобразование Фурье (FFT/NTT)	59
2.4.3 Алгоритм Штрассена и дерева Меркля	60
Выводы по второй главе	62
3 ЭКСПЕРИМЕНТАЛЬНАЯ ПРОВЕРКА ТЕОРЕТИЧЕСКИХ РЕЗУЛЬТАТОВ И ОЦЕНКА ЭФФЕКТИВНОСТИ	64
3.1 Проведение машинного эксперимента	65
3.1.1 Программная реализация алгоритмов со счётчиками сравнений	65
3.1.2 Параметры и методика эксперимента	67
3.2 Сопоставление теоретических и эмпирических результатов.....	68
3.2.1 Верификация формул для сортировки Бэтчера	68
3.2.2 Верификация формул для классических сортировок	71
3.2.3 Сводный сравнительный анализ алгоритмов	73
3.3 Практическая значимость результатов	75
3.3.1 Рекомендации по выбору алгоритма сортировки.....	75
3.3.2 Применимость результатов в параллельных и криптографических системах.....	77
Выводы по третьей главе	78
ЗАКЛЮЧЕНИЕ	80
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	84
ПРИЛОЖЕНИЯ.....	87

СПИСОК ТЕРМИНОВ И СОКРАЩЕНИЙ

В настоящей итоговой аттестационной работе применяют следующие сокращения и обозначения:

Амортизированный анализ – метод оценки сложности, при котором вычисляется средняя стоимость одной операции в произвольной последовательности из n операций, даже если отдельные операции в этой последовательности могут иметь существенно большую стоимость.

Асимптотические обозначения – система обозначений O , Ω , Θ , o , ω , описывающая поведение функции $f(n)$ при $n \rightarrow \infty$ с точностью до константных множителей и слагаемых низшего порядка.

Битоническая последовательность – конечная числовая последовательность, которую можно разбить на две части так, что первая не убывает, а вторая не возрастает, либо являющаяся циклическим сдвигом такой последовательности.

Вероятностный анализ – метод определения математического ожидания времени работы алгоритма при заданном распределении входных данных или при использовании случайных выборов внутри алгоритма.

Вес Хэмминга – количество единиц в двоичной записи целого неотрицательного числа.

Гармоническое число – величина $H_n = \sum_{k=1}^n \frac{1}{k}$, имеющая асимптотическое представление $H_n = \ln n + \gamma + O\left(\frac{1}{n}\right)$, где γ — постоянная Эйлера–Маскерони.

Глубина сортировочной сети — длина наибольшей по числу последовательно зависимых компараторов цепочки в сети; соответствует минимальному числу параллельных тактов при неограниченном числе процессоров.

Дерево Меркля – полное двоичное дерево, листья которого представляют собой криптографические хеши блоков данных, а каждый внутренний узел — хеш от конкатенации хешей его непосредственных потомков.

Дерево решений – двоичное дерево, моделирующее работу алгоритма сортировки сравнениями: во внутренних узлах производятся сравнения пар элементов, а листья соответствуют возможным перестановкам входной последовательности.

Компаратор – элементарный функциональный блок сортировочной сети, принимающий на входе два значения и выдающий их в отсортированном порядке: меньшее – на выход с меньшим индексом, большее — с большим.

Метод раскрутки рекуррентного соотношения – способ решения рекуррентного соотношения путём последовательной подстановки определения функции в само себя до достижения базового случая с последующим суммированием полученной конечной суммы.

Операция «бабочки» – элементарное преобразование пары комплексных величин (E_k, O_k) в пару $(E_k + \omega^k O_k, E_k - \omega^k O_k)$, составляющее шаг объединения результатов в алгоритме быстрого преобразования Фурье.

Парадигма «разделяй и властвуй» — метод проектирования алгоритмов, при котором исходная задача рекурсивно разбивается на несколько подзадач того же типа меньшего размера, решения которых затем комбинируются в решение исходной задачи.

Рекуррентное соотношение – уравнение, выражающее значение функции $T(n)$ через её значения на меньших аргументах и некоторую явно заданную функцию $f(n)$

Сортировочная сеть – заранее заданная для входов фиксированной длины n конечная последовательность компараторов, порядок следования и пары индексов которой не зависят от значений входных данных.

Телескопическое суммирование – техника преобразования рекуррентного соотношения в конечную сумму, в которой соседние члены взаимно сокращаются, оставляя лишь крайние.

Формула Стирлинга - асимптотическое приближение факториала используемое, в частности, при доказательстве нижней границы для сортировки сравнениями.

Чётно-нечётная сортировка слиянием - сортировочная сеть, предложенная К. Э. Бэтчером в 1968 году, основанная на рекурсивном слиянии двух отсортированных половин входа с независимой обработкой элементов на чётных и нечётных позициях.

Число инверсий – количество пар индексов (i, j) , удовлетворяющих условиям $i < j$ и $a_i > a_j$ в массиве a ; служит мерой отклонения массива от отсортированного состояния.

AKS – сортировочная сеть Ajtai–Komlós–Szemerédi глубины $\theta(\log n)$ (1983 год).

DSA – Digital Signature Algorithm, алгоритм цифровой подписи.

ECC – Elliptic Curve Cryptography, криптография на эллиптических кривых.

FFT – Fast Fourier Transform, быстрое преобразование Фурье.

GCD – Greatest Common Divisor, наибольший общий делитель (см. НОД).

GMP – GNU Multiple Precision Arithmetic Library, библиотека арифметики произвольной точности.

GPU – Graphics Processing Unit, графический процессор.

MPC – Multi-Party Computation, протоколы многосторонних защищённых вычислений.

NIST – National Institute of Standards and Technology, Национальный институт стандартов и технологий США.

NTRU – постквантовая криптосистема на основе полиномиальных решёток.

NTT – Number Theoretic Transform, теоретико-числовое преобразование, аналог FFT в конечном поле $\mathbb{Z}_q$.

RSA – Rivest–Shamir–Adleman, асимметричная криптосистема, основанная на сложности задачи факторизации.

IMDS – Single Instruction, Multiple Data, класс параллельной архитектуры «одна инструкция – множество данных».

SPHINCS+ – постквантовая схема цифровой подписи, построенная на деревьях Меркля.

XMSS – eXtended Merkle Signature Scheme, расширенная схема цифровой подписи на деревьях Меркля.

НОД – наибольший общий делитель двух целых чисел.

ВВЕДЕНИЕ

Современный этап развития информационных технологий характеризуется стремительным ростом объёмов обрабатываемых данных и повсеместным распространением параллельных и распределённых вычислительных систем. Алгоритмы сортировки относятся к числу наиболее фундаментальных операций в информатике и применяются практически во всех прикладных областях: при управлении базами данных и файловыми системами, предобработке данных в алгоритмах машинного обучения, организации эффективного поиска, построении индексов, обеспечении целостности данных в криптографических системах [2, 8–11]. Вместе с ростом объёмов данных ужесточаются и требования к производительности сортировки, в особенности в условиях параллельного исполнения и работы в режиме реального времени, что обуславливает необходимость строгой, математически обоснованной оценки эффективности алгоритмов.

Основным математическим аппаратом такой оценки служит асимптотический анализ, оперирующий обозначениями O , Ω и Θ . Он позволяет абстрагироваться от конкретной программной реализации и аппаратных особенностей и описать поведение алгоритма через скорость роста функции сложности при увеличении размера входных данных $n \rightarrow \infty$. Для рекурсивных алгоритмов центральное место в этом аппарате занимает метод решения рекуррентных соотношений, в первую очередь в форме раскрутки — последовательной подстановки соотношения в само себя с последующим суммированием. Данный метод лежит в основе классических доказательств асимптотической сложности сортировки слиянием, быстрой сортировки, умножения Карацубы, быстрого преобразования Фурье и целого ряда других алгоритмов парадигмы «разделяй и властвуй» [2, 4, 19, 21].

Особое положение среди алгоритмов сортировки занимают сортировочные сети - детерминированные вычислительные структуры, в которых

последовательность сравнений и пары сравниваемых индексов не зависят от значений входных данных. Важнейшим представителем этого класса является чётно-нечётная сортировка слиянием (odd-even mergesort), предложенная К. Э. Бэтчером в 1968 году [3]. Обладая числом компараторов порядка $\Theta(n \log^2 n)$ и глубиной сети $\Theta(\log^2 n)$. Данный алгоритм демонстрирует высокий уровень внутреннего параллелизма и сохраняет актуальность в задачах аппаратной реализации, сортировки на SIMD-архитектурах и графических процессорах [18].

Актуальность темы исследования определяется тремя обстоятельствами. Во-первых, возрастающей ролью параллельных сортировок и сортировочных сетей в современных программно-аппаратных системах: сортировка Бэтчера и родственные ей битонические схемы находят непосредственное применение в задачах, где последовательные сортировки оказываются неэффективными [3, 18]. Во-вторых, возрождением интереса к методу решения рекуррентных соотношений в связи с развитием пост-квантовой криптографии, основанной на быстрых полиномиальных умножениях через теоретико-числовое преобразование (NTT), по своей рекурсивной структуре изоморфное рекуррентности, возникающей при анализе чётно-нечётного слияния Бэтчера [25]. В-третьих, отсутствием в доступной учебной и научной литературе систематизированного изложения, в котором единым методом рекуррентных соотношений анализировались бы как сортировочные сети, так и широкий класс классических алгоритмов сортировки и алгоритмы смежных прикладных областей, а границы применимости самого метода были бы явно сформулированы.

Степень разработанности темы. Фундаментальные результаты, связанные с сортировочными сетями и сортировкой Бэтчера, были получены в работе К. Э. Бэтчера (1968) [3] и развиты в классических монографиях Д. Кнута («Искусство программирования», том 3 «Сортировка и поиск», раздел 5.3.4) [18] и Т. Кормена, Ч. Лейзерсона, Р. Ривеста и К. Штайна (CLRS, глава 27 «Sorting Networks») [2]. Общие методы асимптотического анализа — мастер-теорема, метод Акра–Баззи, метод подстановки, метод дерева рекурсии — систематически изложены в

указанных монографиях и в работах Р. Седжвика [15]. Применение метода рекуррентных соотношений к отдельным криптографическим алгоритмам содержится в первоисточниках А. А. Карацубы и Ю. П. Офмана [19], А. Шёнхаге и В. Штрассена [23], Дж. Кули и Дж. Тьюки [21], В. Штрассена [22] и Р. Меркля [24]. Вместе с тем единое изложение, объединяющее анализ сортировки Бэтчера, классических сортировок и криптографических алгоритмов на основе одного и того же методологического аппарата, в доступной учебной литературе представлено фрагментарно. Это определяет необходимость систематизации соответствующих результатов в рамках выпускной квалификационной работы.

Объектом исследования являются алгоритмы сортировки, и прежде всего сортировочная сеть чётно-нечётного слияния Бэтчера, а также связанные с ней классические алгоритмы сортировки и криптографические алгоритмы парадигмы «разделяй и властвуй».

Предметом исследования выступают методы асимптотического анализа сложности указанных алгоритмов, в первую очередь метод решения рекуррентных соотношений через раскрутку и телескопическое суммирование, а также границы его применимости.

Цель работы — разработать и систематически применить единый метод асимптотического анализа, основанный на решении рекуррентных соотношений, к чётно-нечётной сортировке слиянием Бэтчера и родственным ей алгоритмам; получить точные замкнутые формулы их сложности; провести экспериментальную верификацию теоретических результатов на серии машинных экспериментов.

Для достижения поставленной цели в работе решаются следующие задачи:

1. Систематизировать теоретические основы асимптотического анализа алгоритмов сортировки: асимптотические обозначения O , Ω , Θ , o , ω ; метод раскрутки рекуррентных соотношений, мастер-теорему, метод Акра–Баззи, метод подстановки и метод дерева рекурсии; методы прямого подсчёта, вероятностного и амортизированного анализа.
2. Провести полный математический анализ сложности сортировки Бэтчера: записать и решить методом раскрутки рекуррентные соотношения для

числа компараторов операции слияния $M(n)$, полной сортировки $S(n)$ и глубины сети $d_S(n)$; получить точные замкнутые формулы и их асимптотические оценки.

3. Применить тот же метод к классическим алгоритмам сортировки (Merge Sort, Quick Sort, Bitonic Sort, Insertion Sort, Heap Sort) и провести сравнительный анализ с сортировкой Бэтчера по асимптотической сложности, структуре рекуррентности и области применимости.
4. Исследовать границы применимости метода решения рекуррентных соотношений: сформулировать необходимые условия его применения, выявить алгоритмы, для которых требуются альтернативные методы анализа (вероятностный, амортизированный, комбинаторный, прямое суммирование), и обосновать выбор этих методов.
5. Распространить метод на криптографические алгоритмы, имеющие рекурсивную структуру, родственную сортировочным сетям: быстрое преобразование Фурье и теоретико-числовое преобразование, умножение Карацубы и Тома–Кука, алгоритм Штрассена, получить асимптотические оценки их сложности и выявить структурные аналогии с сортировкой Бэтчера.
6. Разработать программную реализацию рассматриваемых алгоритмов сортировки на языке Python с подсчётом числа сравнений; провести машинный эксперимент и экспериментально верифицировать полученные замкнутые формулы на серии размеров входа $n = 2^k$.

Методы исследования. В работе применяются: метод решения рекуррентных соотношений через раскрутку и телескопическое суммирование; мастер-теорема и метод Акра–Баззи; метод математической индукции для проверки полученных замкнутых формул; принцип 0–1 для доказательства корректности сортировочных сетей; методы формального логического доказательства; анализ научной и учебной литературы; программное моделирование алгоритмов сортировки с прямым подсчётом

числа сравнений и статистическая обработка результатов машинного эксперимента.

Теоретическую основу работы составляют положения теории алгоритмов и анализа сложности вычислений, теория сортировочных сетей, аппарат асимптотических обозначений, методы решения рекуррентных соотношений, информационно-теоретический подход к доказательству нижних границ сортировки сравнениями, а также результаты, изложенные в трудах К. Э. Бэтчера, Д. Кнута, Т. Кормена, Ч. Лейзерсона, Р. Ривеста, К. Штайна, Р. Седжвика, А. А. Карацубы, Ю. П. Офмана, А. Шёнхаге, В. Штрассена, Дж. Кули, Дж. Тьюки и Р. Меркля [2, 3, 15, 18–24].

Методическую основу работы образует последовательная схема исследования: формальное описание алгоритма; построение рекуррентного соотношения для его функции сложности; решение рекуррентного соотношения методом раскрутки; проверка полученной формулы методом математической индукции; извлечение асимптотической оценки; сопоставление с известными нижними границами и с результатами машинного эксперимента. Такая схема применяется как к сортировке Бэтчера, так и к классическим алгоритмам сортировки и ряду алгоритмов, используемых в криптографических системах.

Научная новизна работы заключается в следующем:

1. В рамках единой работы на общем методическом основании - методе решения рекуррентных соотношений - проведён сквозной анализ сортировки Бэтчера, родственных ей сортировочных сетей, классических алгоритмов сортировки и применяемых в криптографии алгоритмов «разделяй и властвуй».
2. Сформулированы и явно выделены необходимые условия применимости метода раскрутки рекуррентностей — детерминированность размеров подзадач, наличие рекурсивной структуры, явная выразимость функции

склеивающей работы; на их основе классифицированы алгоритмы сортировки по возможности применения метода.

3. Получены и экспериментально подтверждены точные формулы:

$$M(n) = \frac{n(\log^2 n - 1)}{2} + 1; S(n) = \frac{n \log^2 n (\log^2 n - 1)}{4} + n - 1$$
$$d_{S(n)} = \frac{\log^2 n (\log^2 n + 1)}{2}$$

Основные результаты работы состоят в следующем:

1. Систематизированы методы асимптотического анализа алгоритмов сортировки и показано, какие из них применимы к рекурсивным алгоритмам, а какие — к итеративным, вероятностным и адаптивным алгоритмам.
2. Получены точные формулы числа компараторов операции чётно-нечётного слияния, полной сортировки Бэтчера и глубины соответствующей сортировочной сети.
3. Выполнено сравнение сортировки Бэтчера с классическими алгоритмами сортировки по асимптотической сложности, структуре рекуррентности и области практического применения.
4. Сформулированы границы применимости метода рекуррентных соотношений и показано, почему для ряда алгоритмов необходимы альтернативные методы анализа.
5. Показана применимость того же методического подхода к криптографическим алгоритмам парадигмы «разделяй и властвуй».
6. Проведена экспериментальная проверка теоретических формул с использованием программной реализации алгоритмов сортировки со счётчиками сравнений.

Теоретическая значимость работы состоит в уточнении и систематизации методологии асимптотического анализа сортировочных сетей, а также в обосновании единого методического подхода к анализу широкого класса алгоритмов парадигмы «разделяй и властвуй» — от сортировочных сетей до алгоритмов вычислительной алгебры и криптографии.

Практическая значимость результатов определяется возможностью их использования при: проектировании параллельных сортировок на графических процессорах (GPU), программируемых логических интегральных схемах (FPGA) и других архитектурах с высокой степенью параллелизма; реализации постквантовых криптографических схем CRYSTALS-Kyber и CRYSTALS-Dilithium, опирающихся на теоретико-числовое преобразование (NTT), структурно аналогичное сортировке Бэтчера;

Апробация результатов исследования выполнена посредством машинного эксперимента: разработаны программные реализации рассматриваемых алгоритмов сортировки со встроенными счётчиками сравнений, проведены серии запусков на входах различных размеров, выполнено сопоставление эмпирических значений с теоретическими замкнутыми формулами. Полученные результаты подтверждают корректность выведенных оценок для сортировки Бэтчера и согласуются с известными асимптотическими характеристиками классических алгоритмов сортировки.

Структура и объём работы. Магистерская диссертация состоит из списка терминов и сокращений, введения, трёх глав, заключения, списка использованных источников и приложений. В первой главе «Теоретические основы асимптотического анализа алгоритмов сортировки» рассмотрены алгоритмы сортировки и их классификация, понятие сортировочной сети и её основные свойства, чётно-нечётная сортировка слиянием Бэтчера; приведены асимптотические обозначения, методы решения рекуррентных соотношений, прямой подсчёт, вероятностный и амортизированный анализ; доказана

информационно-теоретическая нижняя граница $\Omega(n \log n)$ для сортировки сравнениями; сформулирована постановка задачи исследования. Во второй главе «Математический анализ сложности алгоритмов сортировки методом рекуррентных соотношений» проведён полный вывод асимптотической сложности сортировки Бэтчера (операция слияния, полная сортировка, глубина сети); тот же метод последовательно применён к классическим алгоритмам сортировки; установлены и обоснованы границы применимости метода; на его основе проанализированы родственные криптографические алгоритмы — умножение Карацубы и Тома–Кука, быстрое преобразование Фурье, алгоритм Штрассена, дерева Меркля.

В третьей главе «Экспериментальная проверка теоретических результатов и оценка эффективности» представлены программная реализация рассматриваемых алгоритмов со счётчиками сравнений, методика и условия проведения машинного эксперимента, результаты верификации полученных замкнутых формул.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АСИМПТОТИЧЕСКОГО АНАЛИЗА АЛГОРИТМОВ СОРТИРОВКИ

1.1 Обзор предметной области

В настоящем разделе рассматриваются базовые понятия, необходимые для формального описания и математического анализа алгоритмов сортировки: определение задачи сортировки, основные признаки классификации известных алгоритмов, понятие сортировочной сети, а также чётно-нечётная сортировка слиянием К. Э. Бэтчера, являющаяся центральным объектом исследования настоящей магистерской диссертации.

1.1.1 Алгоритмы сортировки и их классификация

Задача сортировки формулируется следующим образом. Пусть задана последовательность элементов $a^0, a^1, \dots, a_n^{-1}$. Требуется построить перестановку π множества $\{0, 1, \dots, n - 1\}$ такую, что выполнено условие неубывания $a_{\{\pi(0)\}} \leq a_{\{\pi(1)\}} \leq \dots \leq a_{\{\pi(n-1)\}}$. Элементы последовательности далее называются ключами, а число n — размером входа. Сортировка является одной из наиболее изученных задач в теории алгоритмов; известны десятки алгоритмов её решения, каждый из которых обладает собственным профилем производительности и областью оптимального применения [2, 9, 11–16].

По природе операций над элементами выделяют:

1. *сортировку сравнениями*, в которой единственной разрешённой операцией над ключами является попарное сравнение «меньше — больше» (сортировки вставками, пузырьковая, слиянием, быстрая, пирамидальная, сортировка Бэтчера и др.);
2. *сортировку без сравнений*, в которой используются дополнительные предположения о структуре ключей — например, что они являются целыми числами из ограниченного диапазона, — позволяющие обойтись

без сравнений как таковых (сортировка подсчётом, поразрядная, карманная).

Для сортировки сравнениями существует информационно-теоретическая нижняя граница $\Omega(n \log n)$, выводимая через дерево решений (п. 1.3) [2, 18]; сортировка без сравнений, напротив, в ряде случаев способна выполняться за линейное время $\Theta(n + k)$, где k — диапазон значений ключа.

По асимптотической временной сложности в худшем случае наиболее распространённые алгоритмы делятся на три группы: квадратичные ($\Theta(n^2)$) — сортировка вставками, пузырьковая, выбором; оптимальные по числу сравнений ($\Theta(n \log n)$) — сортировка слиянием, пирамидальная, быстрая сортировка в среднем случае; субквадратичные, но неоптимальные ($\Theta(n \log^2 n)$) — сортировка Бэтчера, битоническая сортировка, сортировка Шелла с последовательностью Пратта [2, 15, 26].

По характеру использования памяти различают сортировки на месте (in-place), требующие $O(1)$ или $O(\log n)$ дополнительной памяти помимо сортируемого массива (пузырьковая, вставками, выбором, пирамидальная, быстрая), и сортировки с дополнительной памятью, использующие $O(n)$ и более дополнительного пространства (сортировка слиянием, сортировка подсчётом, поразрядная).

По свойству устойчивости алгоритм сортировки называется устойчивым, если в выходной последовательности сохраняется взаимный порядок элементов с равными ключами, и неустойчивым в противном случае. Устойчивость имеет значение при многоключевой сортировке, когда последовательно применяется несколько различных ключей.

По отношению к параллельному исполнению выделяют последовательные и параллельные сортировки. Параллельные сортировки, в свою очередь, характеризуются двумя различными мерами сложности: полной работой $W(n)$ —

числом элементарных операций при последовательном выполнении, и глубиной $D(n)$ - длиной критического пути зависимостей. Такие алгоритмы, как сортировка Бэтчера и битоническая сортировка, обладают существенно меньшей глубиной $\theta(\log^2 n)$, чем работа $\theta(n \log^2 n)$, что делает их особенно эффективными для аппаратных реализаций.

Таблица 1.1 Сравнительные характеристики основных алгоритмов сортировки

Алгоритм	Худший случай	Средний случай	Устойчивость	Зависимость от данных
Пузырьковая	$\theta(n^2)$	$\theta(n^2)$	устойчивая	да
Вставками	$\theta(n^2)$	$\theta(n^2)$	устойчивая	да
Выбором	$\theta(n^2)$	$\theta(n^2)$	неустойчивая	да
Слиянием	$\theta(n \log n)$	$\theta(n \log n)$	устойчивая	да
Быстрая	$\theta(n^2)$	$\theta(n \log n)$	неустойчивая	да
Пирамидальная	$\theta(n \log n)$	$\theta(n \log n)$	неустойчивая	да
Битоническая	$\theta(n \log^2 n)$	$\theta(n \log^2 n)$	неустойчивая	нет
Бэтчера	$\theta(n \log^2 n)$	$\theta(n \log^2 n)$	неустойчивая	нет
Подсчётом	$\theta(n + k)$	$\theta(n + k)$	устойчивая	нет (без сравнений)
Поразрядная	$\theta(d(n + k))$	$\theta(d(n + k))$	устойчивая	нет (без сравнений)

В контексте настоящей работы ключевыми являются два признака: зависимость от данных и структура рекурсии. Именно эти признаки определяют применимость основного математического инструмента — метода решения рекуррентных соотношений — к анализу конкретного алгоритма (подробнее см. п. 1.2 и главу 2).

1.1.2 Сортировочные сети: определение и свойства

Сортировочная сеть - это специальный класс алгоритмов сортировки, в которых последовательность выполняемых операций сравнения и пары сравниваемых позиций строго фиксированы и не зависят от значений входных данных. Концепция сортировочных сетей была введена в середине 1960-х годов в работе К. Э. Бэтчера и получила классическое изложение в монографии Д. Кнута «Искусство программирования» (том 3, раздел 5.3.4) [18].

Определение 1.1. Компаратором называется элементарный функциональный блок с двумя входами x и y и двумя выходами $x' = \min(x, y)$ и $y' = \max(x, y)$. В схемной нотации сортировочных сетей компаратор изображается двумя горизонтальными параллельными линиями (провода), соединёнными вертикальным отрезком, указывающим пару сравниваемых позиций.

Определение 1.2. Сортировочной сетью для входов длины n называется конечная последовательность компараторов $C_1, C_2, \dots, C_s$, каждый из которых C_k задаётся упорядоченной парой индексов (i_k, j_k) , $0 \leq i_k < j_k \leq n - 1$. При подаче на вход последовательности $a^0, a^1, \dots, a_n^{-1}$ сеть последовательно выполняет операции сравнения–обмена: для каждого C_k значения $a_{\{i_k\}}$ и $a_{\{j_k\}}$ заменяются на $\min(a_{\{i_k\}}, a_{\{j_k\}})$ и $\max(a_{\{i_k\}}, a_{\{j_k\}})$ соответственно. Сеть называется сортировочной, если для любой входной последовательности её выходная последовательность оказывается неубывающей.

Ключевая особенность сортировочной сети состоит в том, что ни общее число компараторов, ни пары индексов, ни порядок их выполнения *не зависят* от значений ключей - они определяются исключительно длиной входа n . Этим сортировочная сеть принципиально отличается от классических алгоритмов сортировки сравнениями, в которых выбор следующей операции (сравнения, обмена, рекурсивного вызова) диктуется результатами предыдущих сравнений.

С каждой сортировочной сетью связаны две фундаментальные характеристики сложности.

Определение 1.3. Размер сети $S(n)$ - общее число компараторов в сети. Размер определяет сложность последовательного исполнения: если каждая операция сравнения–обмена выполняется за единичное время, полное время работы сети составляет $S(n)$.

Определение 1.4. Глубиной сети $d(n)$ называется длина наибольшего по числу последовательно зависимых компараторов ориентированного пути в графе зависимостей сети. Компараторы, работающие с непересекающимися парами индексов, могут быть исполнены одновременно, поэтому глубина сети определяет параллельное время работы сети при наличии n процессоров.

Таким образом, размер $S(n)$ характеризует «площадь» сети, а глубина $d(n)$ - длину её критического пути. Из определения непосредственно следует неравенство $d(n) \leq S(n)$, и для любой сортировочной сети справедливы обе нижние границы: $S(n) = \Omega(n \log n)$ как следствие общей нижней границы для сортировки сравнениями (п. 1.3) и $d(n) = \Omega(\log n)$, поскольку каждый входной ключ должен быть «связан» хотя бы с одним компаратором, а бинарное дерево зависимостей имеет глубину не менее $\log^2 n$ сортировочные сети обладают рядом существенных прикладных свойств.

1. Детерминированность. Число операций точно фиксировано и известно заранее. Это обеспечивает постоянное время исполнения независимо от входных данных, что критически важно для криптографических применений, где утечка информации через зависимости времени исполнения от ключа является угрозой безопасности.
2. Высокая степень параллелизма. Отношение $\frac{S(n)}{d(n)}$ для сортировки Бэтчера и битонической сортировки составляет $\Theta(n)$, что означает возможность одновременного выполнения порядка $\frac{n}{2}$ компараторов на каждом

параллельном такте. Это делает сортировочные сети удобным объектом для реализации на программируемых логических интегральных схемах (FPGA), графических процессорах (GPU) и в SIMD-архитектурах.

3. Модульность и регулярность структуры. Сортировочные сети строятся из одного типа простых блоков (компараторов) по рекурсивной схеме, что упрощает и аппаратную реализацию, и формальное доказательство корректности.

Среди многочисленных известных сортировочных сетей выделяются три теоретически значимых семейства:

1. Сеть чётно-нечётного слияния Бэтчера (1968), обладающая размером $\Theta(n \log^2 n)$ и глубиной $\Theta(\log^2 n)$, — центральный объект настоящего исследования;
2. Битоническая сортировка Бэтчера с теми же асимптотическими характеристиками, но отличающаяся структурой рекуррентности;
3. Сеть AKS, предложенная в 1983 году М. Айтаи, Я. Комлошем и Э. Семереди, имеющая асимптотически оптимальные размер $\Theta(n \log n)$ и глубину $\Theta(\log n)$, но с настолько большими скрытыми константами, что её практическая реализация оказывается нецелесообразной для любых разумных n (по замечанию Д. Кнута, метод Бэтчера остаётся предпочтительным до тех пор, пока n не превосходит суммарный объём памяти всех вычислительных систем на Земле).

Далее в настоящей работе рассматривается первая из указанных сетей — чётно-нечётная сортировка слиянием Бэтчера.

1.1.3 Чётно-нечётная сортировка слиянием Бэтчера

Чётно-нечётная сортировка слиянием (odd-even mergesort) была предложена К. Э. Бэтчером в статье «Sorting networks and their applications», опубликованной в трудах Spring Joint Computer Conference в 1968 году [3]. Исторически это был

первый практически пригодный алгоритм сортировочной сети с размером, существенно меньшим, чем n^2 .

Сортировка Бэтчера построена по классической схеме «сортировка через слияние»: рекурсивно отсортированные половины массива объединяются в единую отсортированную последовательность оригинально устроенной операцией чётно-нечётного слияния. Основная идея, отличающая алгоритм Бэтчера от классической сортировки слиянием, — именно в устройстве операции слияния: задача слияния двух отсортированных половин длины $\frac{n}{2}$ сводится к двум независимым задачам слияния меньшего размера над чётными и нечётными под-последовательностями.

Операция чётно-нечётного слияния (Odd-Even Merge). На вход подаётся последовательность $a_0, a_1, \dots, a_{n-1}$ длины n (где n — степень двойки), состоящая из двух отсортированных половин:

$$a^0 \leq a^1 \leq \dots \leq a_{\{\frac{n}{2}-1\}}, \quad a_{n/2} \leq a_{\{\frac{n}{2}+1\}} \leq \dots \leq a_{\{n-1\}}.$$

На выходе должна быть получена полностью отсортированная последовательность. Алгоритм определяется рекурсивно:

- 1) при $n = 2$ выполняется одно сравнение (a_0, a_1) ;
- 2) при $n > 2$:
 1. рекурсивно применяется операция чётно-нечётного слияния длины $\frac{n}{2}$ к подпоследовательности элементов с чётными индексами $a_0, a_2, a_4, \dots, a_{n-2}$;
 2. рекурсивно применяется операция чётно-нечётного слияния длины $\frac{n}{2}$ к подпоследовательности элементов с нечётными индексами $a_1, a_3, a_5, \dots, a_{n-1}$;
 3. выполняется серия попарных сравнений (a_i, a_{i+1}) для всех нечётных $i = 1, 3, 5, \dots, n-3$.

Содержательный смысл указанных шагов состоит в следующем. В каждой из двух отсортированных половин входа подпоследовательности на чётных и нечётных позициях сами являются отсортированными. Поэтому шаги 1 и 2 сводят задачу слияния длины n к двум независимым задачам слияния длины $n/2$, каждая из которых оперирует элементами только одной чётности. После выполнения этих шагов подпоследовательность чётных позиций оказывается отсортированной как единое целое, и то же справедливо для подпоследовательности нечётных позиций; однако глобальный порядок на всей последовательности ещё не гарантирован — между чётным элементом и непосредственно следующим за ним нечётным возможно локальное нарушение порядка. Заключительная серия попарных сравнений на шаге 3 устраняет эти нарушения.

Алгоритм сортировки (Odd-Even Merge Sort). На вход подаётся произвольная последовательность $a_0, a_1, \dots, a_{n-1}$ длины n (степень двойки). Алгоритм определяется рекурсивно:

- 1) при $n=1$ последовательность считается отсортированной;
- 2) при $n>1$:
 1. рекурсивно отсортировать первую половину $a_0, a_1, \dots, a_{\{\frac{n}{2}-1\}}$
 2. рекурсивно отсортировать вторую половину $a_{\{\frac{n}{2}\}}, a_{\{\frac{n}{2}+1\}}, \dots, a_{n-1}$;
 3. применить операцию чётно-нечётного слияния длины n к полученной последовательности.

При реализации алгоритма в виде сортировочной сети рекурсия полностью «раскрывается» в плоскую последовательность компараторов, а сама сеть для фиксированного n представляет собой статическую схему, не содержащую условных переходов. В силу независимости шагов 1 и 2 друг от друга (они оперируют непересекающимися половинами массива), а также независимости рекурсивных вызовов слияния для чётных и нечётных под-последовательностей, алгоритм обладает высокой степенью параллелизма: глубина сети составляет

$\theta(\log^2 n)$, в то время как размер — $\theta(n \log^2 n)$. Типичные значения размера и глубины сети для ряда n приведены в таблице 1.2.

Таблица 1.2 - Размер и глубина сети чётно-нечётной сортировки слиянием

n	Число компараторов $S(n)$	Глубина сети $d(n)$
4	5	3
8	19	6
16	63	10
64	543	21
256	3839	36
1024	24063	55

Данные значения точно соответствуют замкнутым формулам, вывод которых методом раскрутки рекуррентных соотношений составляет содержание главы 2. Таким образом, чётно-нечётная сортировка слиянием Бэтчера представляет собой рекурсивно определённую сортировочную сеть с существенно меньшей глубиной по сравнению с её размером и потому высокой пригодностью к параллельному и аппаратному исполнению. Её математический анализ сводится к решению системы рекуррентных соотношений для размера операции слияния, размера полной сортировки и глубины сети, что мотивирует переход к следующему разделу — обзору методов асимптотического анализа алгоритмов, в котором систематически излагается соответствующий математический аппарат

1.2 Методы асимптотического анализа алгоритмов

Для описания скорости роста функций, возникающих при анализе алгоритмов, используется язык асимптотических обозначений, введённый Э. Ландау в конце XIX века и адаптированный Д. Кнудом к задачам теоретической информатики. В настоящем подразделе приводятся формальные определения этих обозначений (п. 1.2.1), после чего рассматриваются пять основных методов

получения асимптотических оценок сложности: раскрутка рекуррентных соотношений (п. 1.2.2), мастер-теорема и метод Акра–Баззи (п. 1.2.3), метод подстановки и метод дерева рекурсии (п. 1.2.4), прямой подсчёт, вероятностный и амортизированный анализ (п. 1.2.5).

1.2.1 Асимптотические обозначения

Пусть $f, g : \mathbb{N} \rightarrow \mathbb{R}_+$ — положительные функции натурального аргумента. Для описания поведения функции $f(n)$ при $n \rightarrow \infty$ с точностью до константных множителей используются пять стандартных обозначений.

Определение 1.5 (О-большое; верхняя асимптотическая граница). Говорят, что $f(n) \in O(g(n))$, если

$$\exists c > 0, n_0 \in \mathbb{N} : \forall n \geq n_0, f(n) \leq c \cdot g(n).$$

Определение 1.6 (Омега-большое; нижняя асимптотическая граница). Говорят, что $f(n) \in \Omega(g(n))$, если

$$\exists c > 0, n_0 \in \mathbb{N} : \forall n \geq n_0, f(n) \geq c \cdot g(n).$$

Определение 1.7 (Тета; точная асимптотическая оценка). Говорят, что $f(n) \in \Theta(g(n))$, если одновременно выполнено $f(n) \in O(g(n))$ и $f(n) \in \Omega(g(n))$, т.е.

$$\exists c_1, c_2 > 0, n_0 \in \mathbb{N} : \forall n \geq n_0, c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n).$$

Определение 1.8 (о-малое; строго меньшая скорость роста). Говорят, что $f(n) \in o(g(n))$, если

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

Определение 1.9 (омега-малое; строго большая скорость роста). Говорят, что $f(n) \in \omega(g(n))$, если

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

Обозначения O и Ω связаны симметрией: $f(n) \in O(g(n)) \Leftrightarrow g(n) \in \Omega(f(n))$. Аналогично, $f(n) \in o(g(n)) \Leftrightarrow g(n) \in \omega(f(n))$. Обозначение Θ играет роль асимптотического «равенства»: запись $f(n) \in \Theta(g(n))$ означает, что функции f и g совпадают по скорости роста и различаются не более чем на постоянный множитель. На практике традиционно применяется упрощённая запись $f(n) = O(g(n))$ вместо строгого « $f(n) \in O(g(n))$ ». В настоящей работе, в соответствии со сложившейся в литературе традицией, обе формы записи используются как синонимы.

При оценке времени работы алгоритмов также различают худший случай $T_{\{worst\}}(n) = \max_{\{|x|=n\}} T(x)$, лучший случай $T_{\{best\}}(n) = \min_{\{|x|=n\}} T(x)$ и средний случай $T_{\{avg\}}(n) = E_{\{|x|=n\}}[T(x)]$, где максимум, минимум и математическое ожидание берутся по всем входным данным размера n . Для сортировочных сетей, не зависящих от данных, все три меры совпадают: $T_{\{worst\}}(n) = T_{\{avg\}}(n) = T_{\{best\}}(n) = S(n)$ что является одним из характерных достоинств этого класса алгоритмов.

1.2.2 Метод решения рекуррентных соотношений

Для большинства рекурсивных алгоритмов, и, в частности, для сортировочных сетей, время работы $T(n)$ удовлетворяет рекуррентному соотношению вида

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n), \quad T(n_0) = const, \quad a \geq 1, b > 1$$

где a — число рекурсивных подзадач, b — коэффициент уменьшения размера подзадачи, а $f(n)$ — стоимость не рекурсивной (склеивающей) работы. Решение такого соотношения — задача о нахождении замкнутой формулы либо асимптотической оценки $T(n)$ — является основной задачей настоящей работы применительно к алгоритму Бэтчера и родственным ему алгоритмам.

Методом раскрутки (telescoping, unfolding) называется приём, состоящий в последовательной подстановке определения функции $T(n)$ в само себя до тех пор, пока аргумент рекурсии не достигнет базового значения n_0 , с последующим явным вычислением полученной конечной суммы. Продемонстрируем метод на рекуррентности $T(n) = 2T\left(\frac{n}{2}\right) + f(n)$. Однократная подстановка даёт

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + f(n) = 2\left[2T\left(\frac{n}{4}\right) + f\left(\frac{n}{2}\right)\right] + f(n) \\ &= 4T\left(\frac{n}{4}\right) + 2f\left(\frac{n}{2}\right) + f(n). \end{aligned}$$

После k шагов раскрутки имеет место общая формула

$$T(n) = 2^k \cdot T\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} 2^i \cdot f\left(\frac{n}{2^i}\right). \quad (1.1)$$

Раскрутку останавливают, когда аргумент рекурсии $\frac{n}{2^k}$ становится равным размеру базового случая. Так, если в качестве базы задано $T(2) = \text{const}$ (как у чётно-нечётного слияния Бэтчера), условие $\frac{n}{2^k} = 2$ даёт $k = \ln_2(n) - 1$; если же база равна $T(1) = \text{const}$, то $k = \ln_2(n)$. После подстановки соответствующего значения k первое слагаемое в формуле (1.1) становится явным (произведение степени двойки на известную константу базы), а второе слагаемое — конечной суммой, вычисляемой стандартными средствами (формулы суммы арифметической и геометрической прогрессий, суммы $\sum i$, $\sum \frac{i}{2^i}$ и т. п.).

Метод раскрутки обладает двумя существенными достоинствами по сравнению с мастер-теоремой. Во-первых, он даёт не только асимптотическую оценку, но и точную замкнутую формулу для $T(n)$, что позволяет определить численные значения функции сложности, а не только порядок её роста. Во-вторых, он применим к рекуррентностям, не укладывающимся в шаблон мастер-теоремы; Корректность замкнутой формулы, полученной раскруткой, обычно проверяется

методом математической индукции по k (или по n). Эта проверка входит в стандартную методологию и систематически применяется во всех доказательствах главы 2.

1.2.3 Мастер-теорема и метод Акра–Баззи

Мастер-теорема (Master Theorem) — универсальный инструмент получения асимптотических оценок для рекуррентностей типа «разделяй и властвуй». Она применима к соотношениям вида

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + \Theta(n^c \log^k n), \quad a \geq 1, b > 1, c \geq 0, k \geq 0.$$

Введём критический показатель $p = \log_b a$. Асимптотика решения определяется сравнением c с p по следующим правилам.

Случай 1. При $c < p$ доминирует рекурсивный вклад: $T(n) = \Theta(n^p)$.

Случай 2. При $c = p$ все уровни рекурсии вносят одинаковый вклад: $T(n) = \Theta(n^p \log^{k+1} n)$

Случай 3. При $c > p$ и выполнении условия регулярности $a \cdot f\left(\frac{n}{b}\right) \leq \rho \cdot f(n)$ для некоторой константы $\rho < 1$ и всех достаточно больших n доминирует склеивающая работа: $T(n) = \Theta(n^c \log^k n)$

В качестве иллюстрации рассмотрим сортировку слиянием с рекуррентностью $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$. Здесь $a = 2$, $b=2$, $f(n) = \Theta(n^1 \log^0 n)$, так что $c = 1$, $k = 0$ и $p = \log_2 2 = 1$. Поскольку $c = p$, реализуется случай 2 и $T(n) = \Theta(n^1 \log^{0+1} n) = \Theta(n \log n)$, что совпадает с известным результатом.

Мастер-теорема существенно ускоряет получение асимптотической оценки, однако обладает двумя принципиальными ограничениями. Во-первых, она не даёт замкнутой формулы с явными числовыми константами. Во-вторых, она неприменима к рекуррентностям с подзадачами неравных размеров, с которыми

сталкиваются при анализе быстрой сортировки в среднем случае и ряда криптографических алгоритмов.

Для таких случаев предназначен метод Акра–Баззи (Akra–Bazzi method), предложенный в 1998 году. Он применим к рекуррентностям вида

$$T(n) = \sum_{i=1}^k a_i \cdot T(b_i \cdot n + h_i(n)) + g(n), \quad a_i > 0, 0 < b_i < 1$$

при слабых регулярностных условиях на возмущения h_i и функцию склеивающей работы g . Сначала из уравнения

$$\sum_{i=1}^k a_i b_i^p = 1$$

определяется единственный показатель p , а затем искомая асимптотика вычисляется по формуле

$$T(n) = \Theta\left(n^p \left(1 + \int_1^n \frac{g(u)}{u^{p+1}} du\right)\right)$$

Так, для быстрой сортировки с фиксированным разбиением 1:9 рекуррентность $T(n) = T\left(\frac{n}{10}\right) + T\left(\frac{9n}{10}\right) + \Theta(n)$ при $a_1 = a_2 = 1, b_1 = \frac{1}{10}, b_2 = \frac{9}{10}$ даёт $p=1$ и $T(n) = \Theta(n \log n)$.

В настоящей работе мастер-теорема используется как вспомогательный инструмент быстрой проверки асимптотических оценок, получаемых методом раскрутки; метод Акра–Баззи привлекается в отдельных случаях в главе 2 — преимущественно в качестве контрпримеров применимости мастер-теоремы.

1.2.4 Метод подстановки и метод дерева рекурсии

Оба метода являются альтернативами раскрутке и часто используются для получения и верификации предварительных гипотез об асимптотической оценке.

Метод подстановки (substitution method) состоит из двух этапов:

1. угадывается форма решения (например, на основе метода дерева рекурсии или общих соображений);
2. корректность гипотезы доказывается методом математической индукции по n .

Проиллюстрируем метод на рекуррентности $T(n) = 2T\left(\frac{n}{2}\right) + n$. Выдвигаем гипотезу: $T(n) \leq c n \log_2 n$ для некоторой константы $c > 0$ и всех достаточно больших n . Индукционный шаг: $T(n) = 2T(n/2) + n \leq 2 \cdot c \cdot n/2 \cdot \log_2 n/2 + n = c n (\log_2 n - 1) + n = c n \log_2 n - c n + n$

При $c \geq 1$ получаем $T(n) \leq c n \log_2 n$, что подтверждает гипотезу. База индукции проверяется отдельно для достаточно малых n . Тем самым $T(n) = O(n \log n)$.

Метод дерева рекурсии (recursion tree method) визуализирует процесс раскрутки в виде дерева, каждый узел которого соответствует одному рекурсивному вызову и помечен стоимостью локальной (не рекурсивной) работы. Асимптотика $T(n)$ получается суммированием стоимостей по всем уровням дерева. Для $T(n) = 2T\left(\frac{n}{2}\right) + n$ дерево рекурсии имеет $\log_2 n + 1$ уровень. На уровне i расположено 2^i узлов, каждый соответствует подзадаче размера $\frac{n}{2^i}$ и имеет локальную стоимость $\frac{n}{2^i}$ суммарная стоимость уровня равна $2^i \cdot \left(\frac{n}{2^i}\right) = n$. Итого:

$T(n) = \sum_{i=0}^{\log_2 n} n = n \cdot (\log^2 n + 1) = \theta(n \log n)$, что совпадает с результатом, полученным мастер-теоремой. Метод дерева рекурсии особенно удобен для гипотетического анализа: он позволяет быстро оценить порядок роста, после чего полученная гипотеза строго доказывается методом подстановки.

1.2.5 Прямой подсчёт, вероятностный и амортизированный анализ

Перечисленные ниже методы применяются в ситуациях, когда рассматриваемый алгоритм не укладывается в парадигму рекурсивного разбиения задачи на подзадачи того же типа, и поэтому метод рекуррентных соотношений напрямую неприменим.

Метод прямого подсчёта (direct counting) используется для итеративных алгоритмов. Время работы $T(n)$ представляется в виде конечной суммы $T(n) = \sum_i t_i$, где t_i — стоимость i -й итерации, после чего сумма вычисляется по явным формулам. Классический пример — сортировка вставками, в которой в худшем случае на i -й итерации выполняется i сравнений, и полное число сравнений равно

$$T(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \theta(n^2)$$

Аналогично анализируются пузырьковая сортировка и сортировка выбором. При анализе пирамидальной сортировки, формально итеративной, прямой подсчёт сочетается с применением формулы суммы ряда (см. п. 2.2.4 и п. 2.3).

Вероятностный анализ (probabilistic analysis) применяется, когда размер подзадач зависит от значений входных данных. Оценкой сложности в этом случае служит математическое ожидание времени работы алгоритма по всем возможным входам при заданном распределении. Центральный пример — быстрая сортировка в среднем случае, среднее число сравнений в которой удовлетворяет рекуррентности

$$T(n) = (n - 1) + \frac{2}{n} \sum_{k=0}^{n-1} T(k)$$

содержащей сумму всех предыдущих значений $T(k)$. Такая рекуррентность не попадает под шаблон мастер-теоремы и не допускает прямой раскрутки; её

решение требует специальных алгебраических преобразований (домножение на n , вычитание соседних уравнений, разложение на простые дроби, использование асимптотики гармонических чисел $H_n = \ln n + \gamma + O(\frac{1}{n})$) и приводит к асимптотике $T(n) = 2(n + 1)H_n - 4n \in \Theta(n \log n)$.

Амортизированный анализ (amortized analysis) оценивает не стоимость отдельной операции, а среднюю стоимость одной операции в произвольной последовательности из n операций. Этот метод применим для алгоритмов со сложной адаптивной политикой (в качестве практически значимого примера выступает сортировка Tim Sort, используемая в стандартных библиотеках Python и Java). При амортизированном анализе вводится функция потенциала Φ от состояния структуры данных и доказывается, что сумма амортизированных стоимостей операций мажорирует сумму их действительных стоимостей. Для Tim Sort такой подход позволяет установить верхнюю оценку $\Theta(n \log n)$ на полное время сортировки, несмотря на наличие отдельных «дорогих» слияний.

Таким образом, совокупность рассмотренных методов — метод рекуррентных соотношений, мастер-теорема, метод Акра–Баззи, методы подстановки и дерева рекурсии, прямой подсчёт, вероятностный и амортизированный анализ — образует достаточно широкий инструментарий для асимптотического анализа алгоритмов сортировки. В главе 2 настоящей работы на примере сортировки Бэтчера демонстрируется применение основного инструмента из этого перечня — метода раскрутки рекуррентных соотношений; в разделе, посвящённом границам применимости метода (п. 2.3), показано, какие именно алгоритмы требуют привлечения альтернативных методов из приведённого списка.

1.3 Информационно-теоретическая нижняя граница сортировки сравнениями

Эффективность алгоритма сортировки может быть оценена не только сверху — в виде явной асимптотической формулы для конкретного алгоритма, но и снизу, в

виде универсальной границы, которую не может преодолеть ни один алгоритм, принадлежащий заданному классу. Для алгоритмов сортировки сравнениями - класса, к которому относятся и рассматриваемые в настоящей работе сортировочные сети, - такая граница составляет $\Omega(n \log n)$ сравнений и доказывается с помощью аргумента дерева решений.

Деревом решений для данного алгоритма сортировки сравнениями называется конечное двоичное дерево, устроенное следующим образом:

1. каждому внутреннему узлу сопоставлено сравнение вида « $a_i \leq a_j$ »; в зависимости от его результата происходит переход в левое или правое поддерево;
2. каждому листу сопоставлена выходная перестановка входной последовательности.

Путь от корня до любого листа соответствует одной конкретной трассировке работы алгоритма и однозначно определяет итоговую перестановку. Поскольку для любой из $n!$ возможных входных перестановок алгоритм должен корректно выдать единственный, строго определённый результат, в дереве решений должно присутствовать не менее $n!$ листьев (в противном случае нашлись бы две различные входные перестановки, порождающие один и тот же путь, а значит, один и тот же результат, что нарушило бы корректность).

Высота любого двоичного дерева с L листьями не меньше $\log_2 L$: действительно, в дереве высоты h не более 2^h листьев. Отсюда для дерева решений алгоритма сортировки сравнениями $h \geq \log_2(n!)$.

Так как максимальная высота дерева решений равна максимальному числу сравнений, выполняемых алгоритмом в худшем случае, имеем

$$T_{\text{worst}}(n) \geq \log_2(n!).$$

Используя формулу Стирлинга $n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$, получаем

$$\begin{aligned}\log^2(n!) &= \sum_{k=1}^n \log_2 k = n \log_2 n - n \log_2 e + \frac{1}{2} \log_2(2\pi n) + o\left(\frac{1}{n}\right) \\ &= n \log_2 n - \theta(n) = \theta(n \log n).\end{aligned}$$

Таким образом, для любого алгоритма сортировки сравнениями справедлива асимптотическая нижняя граница

$$T_{\{worst\}}(n) = \Omega(n \log n) \quad (1.2)$$

Сопоставление неравенства (1.2) с асимптотическими оценками конкретных алгоритмов, полученными в главе 2, позволяет выделить два класса:

1. асимптотически оптимальные алгоритмы с $T(n) = \theta(n \log n)$, достигающие нижней границы (сортировка слиянием, пирамидальная сортировка, быстрая сортировка в среднем случае, сеть AKS);
2. асимптотически неоптимальные по числу сравнений с $T(n) = \omega(n \log n)$, в том числе сортировка Бэтчера, битоническая сортировка, сортировка вставками и ряд других.

Неоптимальность сортировки Бэтчера по числу сравнений ($\theta(n \log 2n)$ против нижней границы $\Omega(n \log n)$) компенсируется её высокой параллельной эффективностью: глубина сети составляет $\theta(\log^2 n)$, в то время как для параллельного исполнения асимптотически оптимальных сортировок требуются существенно более сложные схемы координации процессоров, и в наиболее распространённых параллельных реализациях их глубина оказывается не лучше $\theta(\log^2 n)$. Таким образом, при параллельном исполнении сравнительная неоптимальность сортировочных сетей по числу сравнений перестаёт быть существенным недостатком, что и обуславливает практическую значимость алгоритма Бэтчера на протяжении более полувека.

1.4 Постановка задачи исследования

На основании материала п.п. 1.1–1.3 может быть сформулирована следующая постановка задачи настоящей магистерской диссертации.

Пусть задан класс A рекурсивных алгоритмов сортировки и родственных им алгоритмов парадигмы «разделяй и властвуй», для каждого из которых время работы $T(n)$ удовлетворяет рекуррентному соотношению вида

$$T(n) = \sum_{i=1}^k a_i \cdot T\left(\frac{n}{b_i}\right) + f(n), \quad T(n_0) = \text{const}, \quad a_i \geq 1, b_i > 1$$

Требуется:

1. Для сортировочной сети чётно-нечётного слияния Бэтчера — записать рекуррентные соотношения, определяющие число компараторов операции слияния $M(n)$, число компараторов полной сортировки $S(n)$ и глубину сети $d_S(n)$; методом раскрутки получить точные замкнутые формулы для $M(n)$, $S(n)$ и $d_S(n)$ и доказать их корректность методом математической индукции; из замкнутых формул извлечь асимптотические оценки;
2. Для классических алгоритмов сортировки (сортировка слиянием, быстрая сортировка в лучшем, худшем и среднем случаях, битоническая сортировка Бэтчера, сортировка вставками, пирамидальная сортировка) - записать соответствующие рекуррентные соотношения там, где они имеют место, и получить асимптотические оценки единым методом; выявить алгоритмы, для которых метод раскрутки напрямую неприменим, и обосновать выбор альтернативных методов анализа;
3. Сформулировать необходимые условия применимости метода решения рекуррентных соотношений - критерии, которым должен удовлетворять алгоритм, чтобы метод раскрутки давал его асимптотическую оценку без

- привлечения дополнительных приёмов; на их основе классифицировать рассмотренные алгоритмы;
4. Для криптографических алгоритмов парадигмы «разделяй и властвуй» (умножение Карацубы, умножение Тома–Кука, быстрое преобразование Фурье и теоретико-числовое преобразование, алгоритм Штрассена матричного умножения, дерева Меркля) - применить метод рекуррентных соотношений для получения их асимптотических оценок; выявить структурные аналогии с сортировкой Бэтчера, в частности совпадение вида рекуррентности операции слияния и рекуррентности алгоритма Кули–Тьюки быстрого преобразования Фурье;
 5. Для экспериментальной верификации - разработать на языке Python программные реализации рассматриваемых алгоритмов сортировки со встроенными счётчиками числа выполняемых сравнений; провести серию машинных экспериментов на входах размеров $n = 2^k, k = 1, 2, \dots, K$, и подтвердить соответствие эмпирически наблюдаемого числа сравнений замкнутым формулам, полученным в ходе теоретического анализа.

Решение сформулированных задач составляет содержание глав 2 и 3 настоящей работы.

Выводы по первой главе

Проведённый в настоящей главе обзор теоретических основ асимптотического анализа алгоритмов сортировки позволяет сформулировать следующие выводы:

1. Алгоритмы сортировки классифицируются по нескольким независимым признакам - природе операций над элементами, асимптотической временной сложности, характеру использования памяти, устойчивости, зависимости от данных, степени параллелизма. Из перечисленных признаков в контексте методологии асимптотического анализа определяющими являются

зависимость от данных и структура рекурсии: именно они определяют, какой математический аппарат применим к конкретному алгоритму.

2. Сортировочные сети представляют собой особый класс алгоритмов сортировки, в котором последовательность операций не зависит от значений входных данных. Это обеспечивает постоянство времени исполнения, высокую регулярность структуры, пригодность к аппаратной реализации. Двумя основными характеристиками сортировочной сети являются её размер $S(n)$ - число компараторов, определяющее сложность при последовательном исполнении, и глубина $d(n)$ - длина критического пути зависимостей, определяющая параллельное время работы при наличии n процессоров.
3. Чётно-нечётная сортировка слиянием К. Э. Бэтчера (1968) - одна из первых практически пригодных сортировочных сетей с субквадратичным размером. Она построена по схеме «сортировка через слияние» и отличается оригинально устроенной операцией чётно-нечётного слияния, в которой задача слияния двух отсортированных половин сводится к двум независимым задачам слияния над элементами с чётными и нечётными индексами с последующей серией попарных сравнений.
4. Основным математическим аппаратом анализа рекурсивных алгоритмов является метод решения рекуррентных соотношений, в особенности в форме раскрутки, позволяющей получить не только асимптотическую оценку, но и точную замкнутую формулу для функции сложности. В качестве вспомогательных инструментов выступают мастер-теорема, метод Акра-Баззи, метод подстановки, метод дерева рекурсии. Для алгоритмов, не укладывающихся в парадигму «разделяй и властвуй», применяются методы прямого подсчёта, вероятностного и амортизированного анализа.
5. Для сортировки сравнениями справедлива информационно-теоретическая нижняя граница $T_{worst}(n) = \Omega(n \log n)$, доказываемая через дерево решений с применением формулы Стирлинга. Сортировка Бэтчера не достигает этой границы по числу компараторов (её размер составляет $\Theta(n \log^2 n)$), однако обладает существенно меньшей глубиной сети

$\Theta(\log^2 n)$, что и определяет её практическую значимость для параллельных и аппаратных реализаций.

6. На основании проведённого обзора сформулирована постановка задачи исследования (п. 1.4), включающая анализ сортировки Бэтчера, применение метода рекуррентных соотношений к классическим и криптографическим алгоритмам, исследование границ применимости метода и экспериментальную верификацию полученных результатов. Решение этих задач составляет содержание последующих глав настоящей работы.

2 МАТЕМАТИЧЕСКИЙ АНАЛИЗ СЛОЖНОСТИ АЛГОРИТМОВ СОРТИРОВКИ МЕТОДОМ РЕКУРРЕНТНЫХ СООТНОШЕНИЙ

В первой главе были рассмотрены основные понятия теории сортировки, сортировочных сетей и асимптотического анализа. Было установлено, что для рекурсивных алгоритмов центральным инструментом оценки сложности является метод решения рекуррентных соотношений, в частности метод раскрутки. Настоящая глава посвящена последовательному применению этого метода к чётно-нечётной сортировке слиянием Бэтчера, к ряду классических алгоритмов сортировки, а также к алгоритмам, используемым в криптографических системах и вычислительной алгебре.

Основное внимание уделяется не только получению асимптотических оценок, но и выводу точных замкнутых формул там, где это возможно. Такой подход позволяет сравнивать алгоритмы не только на уровне классов сложности, но и по конкретным численным характеристикам: числу компараторов, глубине сети, числу арифметических операций.

2.1 Доказательство асимптотической сложности сортировки Бэтчера

Чётно-нечётная сортировка слиянием Бэтчера является сортировочной сетью, то есть для фиксированного размера входа n она задаёт заранее известную последовательность компараторов [3, 18]. В силу этой особенности число сравнений не зависит от исходного порядка элементов: лучший, средний и худший случаи совпадают. Поэтому при анализе достаточно определить количество компараторов в сети и её глубину.

Для дальнейшего анализа используются следующие обозначения:

1. $M(n)$ — число компараторов, выполняемых операцией чётно-нечётного слияния (Odd-Even Merge) для последовательности длины n ;

2. $S(n)$ — число компараторов полной сортировочной сети Бэтчера для n входов;
3. $d_M(n)$ — глубина сети операции слияния;
4. $d_S(n)$ — глубина полной сортировочной сети.

Далее предполагается, что n является степенью двойки: $n = 2^m$. Это естественное ограничение для рекурсивного определения сортировочной сети Бэтчера. Для произвольных n возможно дополнение массива фиктивными элементами до ближайшей степени двойки, однако в настоящей работе рассматривается классический вариант алгоритма.

2.1.1 Сложность операции слияния (Odd-Even Merge)

Операция чётно-нечётного слияния получает на вход последовательность длины n , состоящую из двух уже отсортированных половин:

$$a_0 \leq a_1 \leq \dots \leq a_{\frac{n}{2}-1}, \quad a_{\frac{n}{2}} \leq a_{\frac{n}{2}+1} \leq \dots \leq a_{n-1}$$

При $n = 2$ операция слияния сводится к одному компаратору между элементами a_0 и a_1 . При $n > 2$ алгоритм выполняет два рекурсивных слияния длины $\frac{n}{2}$: одно для элементов с чётными индексами, другое для элементов с нечётными индексами. После этого выполняется финальная серия компараторов $(a_1, a_2), (a_3, a_4), \dots, (a_{n-3}, a_{n-2})$, то есть всего $\frac{n}{2} - 1$ дополнительных сравнений.

Следовательно, для числа компараторов операции слияния получаем рекуррентное соотношение

$$M(n) = 2M\left(\frac{n}{2}\right) + \frac{n}{2} - 1, \quad M(2) = \theta(1). \quad (2.1)$$

Пусть $n = 2^m$ и $M_m = M(2^m)$. Тогда соотношение (2.1) принимает вид

$$M_m = 2M_{m-1} + 2^{m-1} - 1, \quad M_1 = 1 \quad (2.2)$$

Решим (2.2) методом раскрутки. Первые шаги имеют вид:

$$M_m = 2M_{m-1} + 2^{m-1} - 1,$$

$$\begin{aligned} M_m &= 2(2M_{m-2} + 2^{m-2} - 1) + 2^{m-1} - 1 \\ &= 4M_{m-2} + 2 \cdot 2^{m-2} + 2^{m-1} - 3. \end{aligned}$$

Так как $2 \cdot 2^{m-2} = 2^{m-1}$, получаем

$$M_m = 4M_{m-2} + 2 \cdot 2^{m-1} - 3.$$

После k шагов раскрутки:

$$M_m = 2^k M_{m-k} + k \cdot 2^{m-1} - (2^k - 1) \quad (2.3)$$

Проверим формулу (2.3) индукцией по k . При $k = 1$ она даёт $2M_{m-1} + 2^{m-1} - 1$, что совпадает с исходной рекуррентностью. Пусть формула верна для некоторого k . Тогда $M_m = 2^k M_{m-k} + k2^{m-1} - 2^k + 1$. Подставляя в неё рекуррентность для M_{m-k} , получаем:

$$M_m = 2^k(2M_{m-k-1} + 2^{m-k-1} - 1) + k2^{m-1} - 2^k + 1,$$

$$M_m = 2^{k+1}M_{m-k-1} + 2^k 2^{m-k-1} + k2^{m-1} - 2^k - 2^k + 1.$$

Так как $2^k 2^{m-k-1} = 2^{m-1}$, имеем

$$M_m = 2^{k+1}M_{m-k-1} + (k+1)2^{m-1} - 2^{k+1} + 1.$$

Следовательно, формула верна и для $k+1$.

Для достижения базового случая M_1 необходимо положить $m-k=1$, то есть $k=m-1$. Подставляя это значение в (2.3), получаем

$$M_m = 2^{m-1}M_1 + (m-1)2^{m-1} - 2^{m-1} + 1.$$

Так как $M_1 = 1$,

$$M_m = 2^{m-1} + (m-1)2^{m-1} - 2^{m-1} + 1 = (m-1)2^{m-1} + 1.$$

Возвращаясь к переменной n , где $n = 2^m$ и $m = \log_2 n$, получаем точную формулу:

$$M(n) = \frac{n}{2} (\log_2 n - 1) + 1. \quad (2.4)$$

Отсюда непосредственно следует асимптотическая оценка

$$M(n) = \frac{n \log_2 n}{2} - \frac{n}{2} + 1 \in \theta(n \log n). \quad (2.5)$$

Таким образом, операция чётно-нечётного слияния Бэтчера имеет ту же асимптотику, что и классическое линейно-логарифмическое слияние в рекурсивных схемах, однако её структура полностью регулярна и представима в виде сортировочной сети.

2.1.2 Сложность полной сортировки

Полная сортировка Бэтчера строится рекурсивно: две половины входа длины $\frac{n}{2}$ сортируются независимо, после чего к ним применяется операция чётно-нечётного слияния длины n . Поэтому число компараторов полной сети удовлетворяет рекуррентности

$$S(n) = 2S\left(\frac{n}{2}\right) + M(n), \quad S(1) = \theta(0). \quad (2.6)$$

Подставляя формулу (2.4) для $M(n)$, имеем

$$S(n) = 2S\left(\frac{n}{2}\right) + \frac{n \log_2 n}{2} - \frac{n}{2} + 1. \quad (2.7)$$

Пусть $n = 2^m$ и $S_m = S(2^m)$. Тогда

$$\begin{aligned} S_m &= 2S_{m-1} + m2^{m-1} - 2^{m-1} + 1 \\ &= 2S_{m-1} + (m-1)2^{m-1} + 1. \end{aligned} \quad (2.8)$$

Разделим обе части (2.8) на 2^m :

$$S_m/(2^m)$$

$$= \frac{S_{m-1}}{2^{m-1}} + \frac{m-1}{2} + \frac{1}{2^m}.$$

Обозначим $t_m = \frac{S_m}{2^m}$. Так как $S_0 = 0$, то $t_0 = 0$. Получаем

$$t_m = t_{m-1} + \frac{m-1}{2} + \frac{1}{2^m}. \quad (2.9)$$

Раскручивая (2.9), получаем

$$t_m = \sum_{i=1}^m \left(\frac{i-1}{2} + \frac{1}{2^i} \right) = \frac{1}{2} \sum_{i=1}^m (i-1) + \sum_{i=1}^m \frac{1}{2^i}.$$

Вычислим суммы:

$$\sum_{i=1}^m (i-1) = 0 + 1 + 2 + \dots + (m-1) = \frac{m(m-1)}{2},$$

$$\sum_{i=1}^m \frac{1}{2^i} = 1 - \frac{1}{2^m}$$

Возвращаясь к $S_m = t_m 2^m$, получаем

$$S_m = 2^m \left(\frac{m(m-1)}{4} + 1 - \frac{1}{2^m} \right) = \frac{m(m-1)}{4} 2^m + 2^m - 1.$$

Так как $2^m = n$ и $m = \log_2 n$, точная формула для числа компараторов полной сортировки Бэтчера имеет вид

$$S(n) = \frac{n \log_2 n (\log_2 n - 1)}{4} + n - 1. \quad (2.10)$$

Доминирующим слагаемым в (2.10) является $\frac{1}{4} n \log_2^2 n$, поэтому

$$S(n) \in \Theta(n \log^2 n). \quad (2.11)$$

Для проверки формулы (2.10) приведём несколько значений:

Таблица 2.1 — Число компараторов сортировки Бэтчера

n	$\log_2 n$	$S(n)$
4	2	5
16	4	63
64	6	543
256	8	3839
1024	10	24063

Полученная оценка показывает, что сортировка Бэтчера асимптотически уступает оптимальным алгоритмам сортировки сравнениями по числу компараторов, поскольку нижняя граница для этого класса равна $\Omega(n \log n)$. Однако это компенсируется регулярностью структуры и высокой степенью параллелизма.

2.1.3 Глубина сети и параллельная сложность

Для сортировочных сетей важно анализировать не только число компараторов, но и глубину сети. Компараторы, работающие с непересекающимися парами элементов, могут выполняться одновременно. Поэтому глубина сети определяет время параллельного исполнения при наличии достаточного числа вычислительных устройств.

Пусть $d_M(n)$ — глубина сети операции чётно-нечётного слияния. В операции слияния два рекурсивных вызова для чётной и нечётной подпоследовательности работают с непересекающимися множествами элементов, следовательно, они могут выполняться параллельно. Финальная серия сравнений

$$(a_1, a_2), (a_3, a_4), \dots, (a_{n-3}, a_{n-2})$$

также состоит из попарно независимых компараторов и занимает один параллельный такт. Поэтому

$$d_{M(n)} = d_M\left(\frac{n}{2}\right) + 1, \quad d_{M(2)} = 1. \quad (2.12)$$

Пусть $n = 2^m$ и $d_m = d_M(2^m)$. Тогда

$$d_m = d_{m-1} + 1, \quad d_1 = 1.$$

Раскрутка даёт

$$d_m = d_1 + (m - 1) = m.$$

Следовательно,

$$d_M(n) = \log_2 n. \quad (2.13)$$

Теперь рассмотрим глубину полной сортировки $d_S(n)$. Две половины массива сортируются независимо, поэтому их сортировка может выполняться параллельно за время $d_S\left(\frac{n}{2}\right)$. После завершения обеих сортировок выполняется слияние глубины $d_M(n)$. Отсюда

$$d_S(n) = d_S\left(\frac{n}{2}\right) + d_M(n), \quad d_S(1) = 0. \quad (2.14)$$

Подставляя (2.13), получаем

$$d_S(n) = d_S\left(\frac{n}{2}\right) + \log_2 n. \quad (2.15)$$

Пусть $n = 2^m$ и $D_m = d_S(2^m)$. Тогда $D_m = D_{m-1} + m$, $D_0 = 0$.

Раскручивая:

$$D_m = D^0 + 1 + 2 + \dots + m = \frac{m(m+1)}{2}.$$

Возвращаясь к n , получаем точную формулу

$$d_{S(n)} = \frac{\log_2 n (\log_2 n + 1)}{2}. \quad (2.16)$$

Отсюда

$$d_S(n) \in \theta(\log^2 n). \quad (2.17)$$

Формула (2.16) имеет важное практическое значение. При последовательном исполнении сортировка Бэтчера требует $\theta(n \log_2 n)$ сравнений, однако при наличии достаточного числа параллельных компараторов время исполнения уменьшается до $\theta(\log^2 n)$. Среднее число компараторов, выполняемых на одном параллельном такте, оценивается как

$$P(n) = \frac{S(n)}{d_S(n)} \approx \frac{\frac{n \log_2^2 n}{4}}{\frac{\log_2^2 n}{2}} = \frac{n}{2}.$$

Таким образом, сеть Бэтчера обеспечивает высокий уровень параллельной загрузки: на каждом такте в среднем может работать порядка $\frac{n}{2}$ компараторов.

2.2 Применение метода к классическим алгоритмам сортировки

Полученные выше результаты демонстрируют, что метод раскрутки рекуррентных соотношений даёт точные формулы для сортировочной сети Бэтчера. Далее тот же подход применяется к классическим алгоритмам сортировки. Цель данного раздела - показать, какие алгоритмы анализируются тем же методом напрямую, а какие требуют дополнительных приёмов или альтернативного математического аппарата.

2.2.1 Сортировка слиянием (Merge Sort)

Сортировка слиянием является классическим алгоритмом парадигмы «разделяй и властвуй». Массив длины n делится на две половины, каждая половина рекурсивно сортируется, после чего две отсортированные половины сливаются за линейное время.

Если $T(n)$ обозначает время работы сортировки слиянием, то

$$T(n) = 2T\left(\frac{n}{2}\right) + cn, \quad T(1) = 0, \quad (2.18)$$

Раскроем (2.18):

$$T(n) = 2T\left(\frac{n}{2}\right) + cn,$$

$$T(n) = 4T\left(\frac{n}{4}\right) + 2cn,$$

$$T(n) = 8T\left(\frac{n}{8}\right) + 3cn.$$

После k шагов:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kcn. \quad (2.19)$$

При $k = \log_2 n$ достигается базовый случай $T(1) = 0$, поэтому

$$T(n) = nT(1) + cn \log_2 n = cn \log_2 n.$$

Следовательно:

$$T(n) \in \Theta(n \log n). \quad (2.20)$$

Структура рекуррентности сортировки слиянием совпадает с общей схемой анализа сортировки Бэтчера: две подзадачи половинного размера и явно выраженная стоимость слияния. Отличие состоит в том, что классическое слияние имеет линейную стоимость $\Theta(n)$, а чётно-нечётное слияние Бэтчера само является рекурсивной сетью с точной формулой (2.4).

2.2.2 Быстрая сортировка (Quick Sort)

Быстрая сортировка также относится к алгоритмам «разделяй и властвуй», однако размеры её подзадач зависят от положения опорного элемента [2, 15]. Поэтому анализ существенно различается для худшего, лучшего и среднего случаев.

В худшем случае опорный элемент каждый раз оказывается минимальным или максимальным, и одна из подзадач имеет размер $n-1$, а другая — размер 0. Тогда:

$$T(n) = T(n-1) + cn, \quad T(1) = 0. \quad (2.21)$$

Раскрутка даёт

$$T(n) = T(n-1) + cn = T(n-2) + c(n-1) + cn = \dots = c \sum_{i=2}^n i.$$

Отсюда

$$T(n) = c \left(\frac{n(n+1)}{2} - 1 \right) \in \theta(n^2). \quad (2.22)$$

В лучшем случае опорный элемент каждый раз делит массив пополам. Тогда рекуррентность совпадает с рекуррентностью сортировки слиянием:

$$T(n) = 2T(n/2) + cn, \quad (2.23)$$

и, как было показано выше

$$T(n) \in \theta(n \log n). \quad (2.24)$$

Средний случай требует отдельного анализа. Если считать, что опорный элемент равновероятно занимает любую позицию от 0 до $n-1$, то среднее число сравнений удовлетворяет соотношению

$$T(n) = (n-1) + \frac{2}{n} \sum_{k=0}^{n-1} T(k), \quad T(0) = T(1) = 0. \quad (2.25)$$

Это уже не рекуррентность вида $T(n) = aT(\frac{n}{b}) + f(n)$: она содержит сумму всех предыдущих значений. Решение получается через преобразование к конечно-разностному уравнению. Помножим (2.25) на n :

$$nT(n) = n(n-1) + 2 \sum_{k=0}^{n-1} T(k). \quad (2.26)$$

Для $n-1$:

$$(n-1)T(n-1) = (n-1)(n-2) + 2 \sum_{k=0}^{n-2} T(k). \quad (2.27)$$

Вычитая (2.27) из (2.26), получаем

$$nT(n) = (n+1)T(n-1) + 2(n-1). \quad (2.28)$$

Делим на $n(n+1)$:

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2(n-1)}{n(n+1)}. \quad (2.29)$$

Обозначим $u_n = \frac{T(n)}{n+1}$. Тогда

$$u_n = u_{n-1} + \frac{2(n-1)}{n(n+1)}.$$

После телескопирования и разложения на простые дроби получаем известную формулу

$$T(n) = 2(n+1)H_n - 4n, \quad (2.30)$$

где $H_n = \sum_{i=1}^n \frac{1}{i}$ — n -е гармоническое число. Так как $H_n = \ln n + O(1)$, то

$$T(n) \in \theta(n \log n). \quad (2.31)$$

Таким образом, для быстрой сортировки метод раскрутки напрямую применим в худшем и лучшем случаях, но средний случай требует вероятностного анализа и дополнительных алгебраических преобразований.

2.2.3 Битоническая сортировка (Bitonic Sort)

Битоническая сортировка, как и чётно-нечётная сортировка Бэтчера, является сортировочной сетью. Она основана на понятии битонической последовательности, то есть последовательности, сначала неубывающей, а затем невозрастающей, либо циклического сдвига такой последовательности.

Пусть $B(n)$ — число компараторов в битоническом слиянии длины n . Тогда

$$B(n) = 2B\left(\frac{n}{2}\right) + \frac{n}{2}, \quad B(2) = 1. \quad (2.32)$$

Пусть $n = 2^m$ и $B_m = B(2^m)$. Тогда

$$B_m = 2B_{m-1} + 2^{m-1}, B_1 = 1.$$

Раскрутка даёт

$$B_m = 2^k B_{m-k} + k2^{m-1}.$$

При $k = m - 1$:

$$B_m = 2^{m-1} B_1 + (m - 1)2^{m-1} = m2^{m-1}.$$

Следовательно,

$$B(n) = \frac{n}{2} \log_2 n \in \theta(n \log n). \quad (2.33)$$

Для полной битонической сортировки:

$$S_{B(n)} = 2S_B\left(\frac{n}{2}\right) + B(n) = 2S_B\left(\frac{n}{2}\right) + \frac{n}{2} \log_2 n. \quad (2.34)$$

Переходя к $n = 2^m$ и $s_m = S_B(2^m)$, имеем

$$s_m = 2s_{m-1} + m2^{m-1}, s_0 = 0.$$

Делим на 2^m и обозначаем $t_m = \frac{s_m}{2^m}$:

$$t_m = t_{m-1} + \frac{m}{2}.$$

Тогда

$$t_m = \frac{1}{2} \sum_{i=1}^m i = \frac{m(m+1)}{4},$$

$$S_B(n) = \frac{n \log^2 n (\log^2 n + 1)}{4} \in \Theta(n \log^2 n). \quad (2.35)$$

Битоническая сортировка и сортировка Бэтчера имеют одинаковый асимптотический порядок размера сети, однако их точные формулы различаются. Для сортировки Бэтчера ведущий член также равен $\frac{1}{4} n \log_2^2 n$, но младшие члены отличаются:

$$S(n) = \frac{n \log^2 n (\log^2 n - 1)}{4} + n - 1.$$

2.2.4 Сортировка вставками и пирамидальная сортировка

Сортировка вставками не является алгоритмом «разделяй и властвуй», однако её худший случай можно формально представить рекуррентностью:

$$T(n) = T(n-1) + cn, \quad T(1) = 0. \quad (2.36)$$

Эта запись соответствует тому, что после сортировки первых $n-1$ элементов последний элемент в худшем случае вставляется в начало за линейное число сравнений. Раскрутка даёт

$$T(n) = c \sum_{i=1}^n i = \frac{cn(n+1)}{2} \in \Theta(n^2). \quad (2.37)$$

Однако в данном случае рекуррентность является скорее искусственной формой записи прямого суммирования. Алгоритм фактически выполняет

последовательность итераций, а не рекурсивно делит задачу на подзадачи того же типа.

Пирамидальная сортировка (Heap Sort) состоит из двух фаз: построения максимальной кучи и последовательного извлечения максимального элемента с восстановлением свойства кучи. Эта структура также не порождает рекуррентность вида $T(n) = aT(\frac{n}{b}) + f(n)$. Операция просеивания вниз для узла высоты h имеет сложность

$$T_{\{sift\}}(h) \leq 2h. \quad (2.38)$$

Построение кучи снизу вверх оценивается суммой по уровням дерева:

$$T_{build}(n) \leq \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^{h+1}} 2h \leq n \sum_{n=0}^{\infty} \frac{h}{2^n} = 2n. \quad (2.39)$$

Следовательно, построение кучи выполняется за $O(n)$. Фаза извлечения требует не более

$$T_{extract}(n) = \sum_{j=1}^{n-1} O(\log i) = O(n \log n). \quad (2.40)$$

Итого

$$T(n) = T_{build}(n) + T_{extract}(n) \in \Theta(n \log n). \quad (2.41)$$

Принципиально важно, что для Heap Sort результат получается не методом раскрутки рекуррентного соотношения, а методом прямого суммирования стоимости операций. Это делает пирамидальную сортировку важным примером алгоритма, асимптотически эффективного, но не подходящего под прямое применение основного метода настоящей работы.

2.3 Границы применимости метода рекуррентных соотношений

Предыдущий раздел показывает, что метод рекуррентных соотношений применим не ко всем алгоритмам сортировки одинаково. Для одних алгоритмов он является прямым и естественным инструментом анализа, для других требуется модификация, а для третьих метод фактически неприменим.

2.3.1 Необходимые условия применимости

Для прямого применения метода раскрутки рекуррентных соотношений алгоритм должен удовлетворять трём условиям.

Условие А — рекурсивная структура. Задача размера n должна сводиться к одной или нескольким подзадачам того же типа меньшего размера. Например, сортировка слиянием сводит сортировку массива длины n к сортировке двух массивов длины $\frac{n}{2}$; сортировка Бэтчера сводит сортировку к двум сортировкам половин и чётно-нечётному слиянию.

Условие В — детерминированные размеры подзадач. Размеры подзадач должны определяться только n , а не содержимым входных данных. В сортировке Бэтчера подзадачи всегда имеют размеры $\frac{n}{2}$, поэтому рекуррентность фиксирована. В быстрой сортировке размеры подзадач зависят от положения опорного элемента, что нарушает это условие в среднем случае.

Условие С — явно выраженная склеивающая работа. Работа, выполняемая вне рекурсивных вызовов, должна быть выражена формулой от n . В сортировке слиянием это cn , в чётно-нечётном слиянии Бэтчера — $\frac{n}{2} - 1$, в алгоритме Карацубы — $\Theta(n)$.

Если все три условия выполнены, то сложность алгоритма обычно можно получить по схеме: $T(n) = aT(\frac{n}{b}) + f(n) \Rightarrow T(n) = a^k T(\frac{n}{b^k}) + \sum_{i=0}^{k-1} a^i f(\frac{n}{b^i})$

Именно эта схема использовалась при анализе сортировки Бэтчера, сортировки слиянием, битонической сортировки, и как будет видно в дальнейшем в нахождении асимптотической сложности умножения Карацубы, алгоритмов Tom-Cook и FFT.

2.3.2 Алгоритмы, требующие альтернативных методов анализа

Рассмотрим основные случаи нарушения условий A–C.

Быстрая сортировка в среднем случае нарушает условие B. Позиция опорного элемента зависит от входных данных или случайного выбора. Формально размеры подзадач описываются случайной величиной I_n :

$$T(n) = T(I_n) + T(n - 1 - I_n) + cn.$$

После перехода к математическому ожиданию возникает соотношение

$$ET(n) = cn + \frac{2}{n} \sum_{k=0}^{n-1} ET(k),$$

которое требует вероятностного анализа и дополнительных преобразований.

Пирамидальная сортировка нарушает условие A. Она не делит задачу на рекурсивные подзадачи того же типа, а выполняет построение кучи и последовательное извлечение максимумов. Поэтому её сложность определяется прямым суммированием стоимостей операций siftDown.

Сортировка Шелла нарушает сразу несколько условий. Она выполняет серию сортировок вставками по под-последовательностям с различными шагами $h_1, h_2, \dots, h_t = 1$. Стоимость каждого прохода зависит от выбранной последовательности шагов и от числа инверсий, оставшихся после предыдущих проходов. Поэтому для Shellsort не существует единого простого рекуррентного соотношения; анализ требует комбинаторных и теоретико-числовых методов.

Tim Sort является адаптивным гибридным алгоритмом, применяемым в Python и Java. Он находит естественные серии (runs) во входных данных и затем сливает их по стратегии, зависящей от длин серий. Размеры подзадач и стоимость слияний зависят от входа, поэтому вместо обычной раскрутки применяют амортизированный анализ, в частности метод потенциалов.

Пузырьковая сортировка и сортировка выбором являются итеративными алгоритмами. Их можно искусственно записать в виде $T(n) = T(n - 1) + cn$, но такая запись не отражает реальной рекурсивной структуры, а лишь маскирует прямое суммирование арифметической прогрессии. Сводно причины неприменимости метода можно представить так:

Таблица 2.2 — Причины ограничения метода рекуррентных соотношений

Алгоритм	Нарушенное условие	Причина	Альтернативный метод
Quick Sort, средний случай	В	случайный размер подзадач	вероятностный анализ
Heap Sort	А	нет рекурсивной структуры сортировки	прямое суммирование
Shellsort	А, В, С	зависимость от последовательности шагов и входа	комбинаторный анализ
Tim Sort	В, С	адаптивные серии и стратегия слияния	амортизированный анализ
Bubble Sort, Selection Sort	А	итеративная структура	прямой подсчёт

Следовательно, метод рекуррентных соотношений является мощным, но не универсальным инструментом. Его естественная область применимости — алгоритмы с детерминированной рекурсивной структурой и явно выраженной стоимостью объединения результатов подзадач.

2.4 Применение метода к криптографическим алгоритмам

Метод рекуррентных соотношений используется не только в анализе сортировок. Многие алгоритмы вычислительной алгебры и криптографии имеют структуру «разделяй и властвуй»: большие числа, полиномы или матрицы разбиваются на части, затем для частей рекурсивно решается та же задача, а результаты объединяются. Поэтому тот же аппарат применим к оценке сложности умножения больших чисел, быстрого преобразования Фурье, матричного умножения и деревьев Меркля.

2.4.1 Умножение Карацубы и алгоритм Тоома–Кука

Стандартное умножение двух n -разрядных чисел требует $\theta(n^2)$ элементарных операций. Алгоритм Карацубы уменьшает число рекурсивных умножений половинных чисел с четырёх до трёх.

Представим числа x и y в виде

$$x = x_1 B^{\frac{n}{2}} + x_0, \quad y = y_1 B^{\frac{n}{2}} + y_0,$$

где B — основание системы счисления, а x_0, x_1, y_0, y_1 имеют длину $\frac{n}{2}$.

Наивное разложение требует четырёх произведений:

$$xy = x_1 y_1 B^n + (x_1 y_0 + x_0 y_1) B^{\frac{n}{2}} + x_0 y_0.$$

Карацуба заменяет два перекрёстных произведения одним:

$$z_2 = x_1 y_1, \quad z_0 = x_0 y_0,$$

$$z_1 = (x_1 + x_0)(y_1 + y_0) - z_2 - z_0 = x_1 y_0 + x_0 y_1.$$

Таким образом, рекуррентность имеет вид

$$T(n) = 3T\left(\frac{n}{2}\right) + cn, \quad T(1) = O(1). \quad (2.42)$$

Раскрутка даёт

$$T(n) = 3^k T\left(\frac{n}{2^k}\right) + cn \sum_{i=0}^{k-1} \left(\frac{3}{2}\right)^i. \quad (2.43)$$

При $k = \log_2 n$:

$$3^{\log^2 n} = n^{\log^2 3}$$

а геометрическая сумма имеет тот же порядок. Следовательно,

$$T(n) \in \theta(n^{\log^2 3}) \approx \theta(n^{1.585}). \quad (2.44)$$

Алгоритм Тома–Кука обобщает идею Карацубы. В варианте Тоом-3 каждое число разбивается на три части; произведение рассматривается как произведение двух полиномов степени 2, которое восстанавливается по пяти значениям. Поэтому требуется пять рекурсивных умножений чисел длины $n/3$:

$$T(n) = 5T\left(\frac{n}{3}\right) + cn. \quad (2.45)$$

При $k = \log_3 n$ получаем

$$T(n) \in \theta(n^{\log^3 5}) \approx \theta(n^{1.465}). \quad (2.46)$$

Для общего Тоом- k числа разбиваются на k частей, а число рекурсивных умножений равно $2k - 1$:

$$T(n) = (2k - 1)T\left(\frac{n}{k}\right) + \theta(n),$$

Откуда

$$T(n) \in \Theta(n^{\log_k(2k-1)}). \quad (2.47)$$

И Карацуба, и Тоом-Куок демонстрируют ту же методологическую схему, что и сортировка Бэтчера: рекурсивное разбиение, явная стоимость объединения и получение оценки через раскрутку рекуррентности.

2.4.2 Быстрое преобразование Фурье (FFT/NTT)

Быстрое преобразование Фурье (FFT) и его конечно-полевой аналог - теоретико-числовое преобразование (NTT) лежат в основе быстрых алгоритмов полиномиального умножения. NTT особенно важен для пост-квантовой криптографии, например для схем CRYSTALS-Kyber и CRYSTALS-Dilithium

Дискретное преобразование Фурье последовательности $a_0, \dots, a_{n-1}$ задаётся формулой

$$A_k = \sum_{j=0}^{n-1} a_j \omega^{jk}, \quad k = 0, \dots, n-1,$$

где ω — примитивный корень степени n (в комплексных числах для FFT или в конечном поле для NTT).

Алгоритм Кули–Тьюки разделяет исходную последовательность на элементы с чётными и нечётными индексами:

$$A_k = \sum_{j=0}^{\frac{n}{2}-1} a_{2j} \omega^{2jk} + \omega^k \sum_{j=0}^{\frac{n}{2}-1} a_{2j+1} \omega^{2jk}.$$

Обозначая первые и вторые суммы через E_k и O_k , получаем операции объединения:

$$A_k = E_k + \omega^k O_k, \quad A_{k+n/2} = E_k - \omega^k O_k$$

Эти операции называются «бабочками». На каждом уровне выполняется линейное число таких операций, поэтому

$$T(n) = 2T\left(\frac{n}{2}\right) + cn, \quad T(1) = 0. \quad (2.48)$$

Раскрутка полностью совпадает с анализом сортировки слиянием:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kcn.$$

При $k = \log_2 n$:

$$T(n) = cn \log_2 n \in \theta(n \log n). \quad (2.49)$$

Структурная аналогия между FFT и чётно-нечётным слиянием Бэтчера состоит в том, что оба алгоритма используют разбиение по чётности индексов и имеют рекуррентность с двумя подзадачами половинного размера. Различие заключается в стоимости объединения: у FFT она равна cn , у слияния Бэтчера — $\frac{n}{2} - 1$. В обоих случаях итоговая стоимость соответствующей базовой операции имеет порядок $\theta(n \log n)$.

2.4.3 Алгоритм Штрассена и дерева Меркля

Алгоритм Штрассена для умножения матриц показывает, что метод рекуррентных соотношений применим не только к сортировкам и умножению чисел. Наивное умножение двух матриц $n \times n$ требует $\theta(n^3)$ скалярных умножений. Штрассен разбивает матрицы на четыре блока размера $\frac{n}{2} \times \frac{n}{2}$ и заменяет восемь рекурсивных умножений семью, увеличивая число сложений и вычитаний матриц.

Рекуррентность имеет вид

$$T(n) = 7T\left(\frac{n}{2}\right) + cn^2. \quad (2.50)$$

Раскрутка:

$$T(n) = 7^k T\left(\frac{n}{2^k}\right) + cn^2 \sum_{i=0}^{k-1} \left(\frac{7}{4}\right)^i.$$

При $k = \log_2 n$:

$$7^{\log^2 n} = n^{\log^2 7}.$$

Поскольку $\log_2 7 \approx 2.807 > 2$, доминирует рекурсивный вклад, и

$$T(n) \in \theta(n^{\log^2 7}) \approx \theta(n^{2.807}) \quad (2.51)$$

Деревья Меркля, напротив, дают пример рекуррентности с линейным итогом. Полное двоичное дерево с n листьями строится рекурсивно из двух поддеревьев размера $\frac{n}{2}$, после чего вычисляется хеш корня:

$$T(n) = 2T\left(\frac{n}{2}\right) + 1, \quad T(1) = 1. \quad (2.52)$$

Пусть $n = 2^m$ и $T_m = T(2^m)$. Тогда

$$T_m = 2T_{m-1} + 1.$$

После k шагов:

$$T_m = 2^k T_{m-k} + \sum_{i=0}^{k-1} 2^i = 2^k T_{m-k} + 2^k - 1.$$

Следовательно,

$$T(n) = 2n - 1 \in \theta(n). \quad (2.53)$$

Проверка пути принадлежности элемента дереву Меркля имеет другую природу: путь от листа к корню содержит $\log_2 n$ уровней, поэтому

$$T_{proof}(n) \in \theta(\log n). \quad (2.54)$$

Таким образом, криптографические алгоритмы демонстрируют широкий спектр рекуррентностей: от линейных деревьев Меркля до субквадратичного умножения Карацубы и Тоом-Кука, а также алгоритма Штрассена с показателем $\log_2 7$. Общим для них является наличие рекурсивной структуры, позволяющей применять тот же методический аппарат, что и для сортировочных сетей.

Выводы по второй главе

В настоящей главе проведён математический анализ сложности сортировки Бэтчера, классических алгоритмов сортировки и ряда криптографических алгоритмов методом рекуррентных соотношений. Получены следующие основные результаты.

1. Для операции чётно-нечётного слияния Бэтчера получена точная формула числа компараторов: $M(n) = \frac{n}{2}(\log_2 n - 1) + 1$, откуда следует $M(n) \in \Theta(n \log n)$.
2. Для полной сортировки Бэтчера получена точная формула $S(n) = \frac{n \log^2 n (\log^2 n - 1)}{4} + n - 1$, асимптотическая оценка $S(n) \in \Theta(n \log^2 n)$.
3. Для глубины сортировочной сети Бэтчера получена формула $d_{S(n)} = \frac{\log^2 n (\log^2 n + 1)}{2}$, что даёт параллельную сложность $\Theta(\log^2 n)$ при наличии достаточного числа компараторов.
4. Показано, что метод раскрутки рекуррентных соотношений напрямую применим к сортировке слиянием, битонической сортировке, худшему и лучшему случаям быстрой сортировки, а также формально к сортировке вставками. Для пирамидальной сортировки, среднего случая Quick Sort, Shellsort и Tim Sort требуются альтернативные методы анализа.
5. Сформулированы необходимые условия применимости метода: наличие рекурсивной структуры, детерминированность размеров подзадач и явная выразимость стоимости склеивающей работы.

6. Метод рекуррентных соотношений распространён на криптографические алгоритмы и алгоритмы вычислительной алгебры: умножение Карацубы, Тоом-Кука, FFT/NTT, алгоритм Штрассена и дерева Меркля. Это подтверждает универсальность метода для алгоритмов парадигмы «разделяй и властвуй».

Полученные во второй главе формулы и асимптотические оценки служат теоретической основой для машинной проверки, выполняемой в третьей главе.

3 ЭКСПЕРИМЕНТАЛЬНАЯ ПРОВЕРКА ТЕОРЕТИЧЕСКИХ РЕЗУЛЬТАТОВ И ОЦЕНКА ЭФФЕКТИВНОСТИ

В предыдущей главе были получены теоретические оценки сложности чётно-нечётной сортировки слиянием Бэтчера, битонической сортировки, сортировки слиянием, быстрой сортировки, сортировки вставками и пирамидальной сортировки. Основным результатом для сортировки Бэтчера стали точные замкнутые формулы числа компараторов полной сортировочной сети и глубины сети:

$$S(n) = \frac{(n \log^2 n (\log^2 n - 1))}{4} + n - 1,$$
$$d_{S(n)} = \frac{\log^2 n (\log^2 n + 1)}{2}.$$

Настоящая глава посвящена машинной проверке полученных формул. Экспериментальная часть выполнялась с использованием программных реализаций алгоритмов сортировки на языке Python, в которые были встроены счётчики сравнений. Такой подход позволяет сопоставить теоретически рассчитанное число сравнений с фактическим числом сравнений, выполненных программой на входах различного размера.

Экспериментальная проверка не заменяет математического доказательства, приведённого во второй главе, но выполняет две важные функции. Во-первых, она подтверждает корректность реализации алгоритмов и правильность полученных формул для конкретных значений n . Во-вторых, она позволяет наглядно сравнить темпы роста различных алгоритмов и продемонстрировать практическое различие между классами сложности $\Theta(n \log n)$, $\Theta(n \log^2 n)$ и $\Theta(n^2)$.

3.1 Проведение машинного эксперимента

Эксперимент проводился в среде Jupyter Notebook. Основной файл эксперимента — `experiments/verification.ipynb`. В нём реализованы алгоритмы сортировки со встроенными счётчиками сравнений, заданы теоретические формулы сложности, выполнены серии запусков для входов размера $n = 2^k$ и построены графики сопоставления эмпирических и теоретических результатов.

3.1.1 Программная реализация алгоритмов со счётчиками сравнений

В эксперименте использовались реализации следующих алгоритмов:

1. Чётно-нечётная сортировка слиянием Бэтчера (Odd-Even Merge Sort).
2. Битоническая сортировка (Bitonic Sort).
3. Сортировка слиянием (Merge Sort).
4. Быстрая сортировка (Quick Sort) в худшем и среднем случаях.
5. Сортировка вставками (Insertion Sort).
6. Пирамидальная сортировка (Heap Sort).
7. Сортировка Студжа (Stooge Sort) — как дополнительный пример рекурсивного алгоритма с плохой асимптотикой.

Каждая функция сортировки возвращает пару значений: (отсортированный массив, число сравнений). Для подсчёта сравнений в рекурсивных алгоритмах использовался изменяемый счётчик, передаваемый во внутренние функции рекурсии. Ниже приведён фрагмент реализации чётно-нечётного слияния Бэтчера, используемый в эксперименте:


```

1  def odd_even_merge(arr, lo, hi, comparisons):
2      n = hi - lo
3      if n <= 1:
4          return
5      if n == 2:
6          comparisons[0] += 1
7          if arr[lo] > arr[lo + 1]:
8              arr[lo], arr[lo + 1] = arr[lo + 1], arr[lo]
9          return
10
11     even = [arr[lo + i] for i in range(0, n, 2)]
12     odd = [arr[lo + i] for i in range(1, n, 2)]
13
14     odd_even_merge(even, 0, len(even), comparisons)
15     odd_even_merge(odd, 0, len(odd), comparisons)
16
17     for i in range(len(even)):
18         arr[lo + 2 * i] = even[i]
19     for i in range(len(odd)):
20         arr[lo + 2 * i + 1] = odd[i]
21
22     for i in range(1, n - 1, 2):
23         comparisons[0] += 1
24         if arr[lo + i] > arr[lo + i + 1]:
25             arr[lo + i], arr[lo + i + 1] = arr[lo + i + 1], arr[lo + i]

```

Рисунок 3.1 Фрагмент реализации операции Odd-Even Merge со счётчиком сравнений

Полная сортировка Бэтчера реализована рекурсивно: сначала сортируются две половины массива, затем к ним применяется функция `odd_even_merge`. Важно, что число сравнений в сортировочной сети Бэтчера не зависит от входных данных. Поэтому для проверки формулы $S(n)$ достаточно запустить алгоритм на одном произвольном массиве заданной длины.

Аналогичный подход применён к битонической сортировке. Функции *bitonic_sort_rec* и *bitonic_merge* также используют счётчик сравнений, а число сравнений фиксировано структурой сети.

Для сортировки слиянием счётчик увеличивается в момент сравнения текущих элементов левой и правой отсортированных половин. В отличие от сортировочных сетей, фактическое число сравнений Merge Sort может зависеть от конкретного распределения входных данных, однако в худшем случае для $n=2^m$ оно близко к формуле $T(n) = n \log_2 n - n + 1$.

Для быстрой сортировки в эксперименте реализованы две стратегии выбора опорного элемента:

1. `pivot_strategy='first'` — первый элемент используется как опорный; на уже отсортированном входе это приводит к худшему случаю;
2. `pivot_strategy='random'` — опорный элемент выбирается случайно; результат усредняется по нескольким запускам.

Для сортировки вставками использовался обратно отсортированный массив, поскольку он соответствует худшему случаю и приводит к точному числу сравнений $T(n) = \frac{n(n-1)}{2}$.

Для пирамидальной сортировки подсчитывались сравнения при восстановлении свойства кучи в операции `sift_down`. Поскольку точное число сравнений Heap Sort зависит от структуры кучи и входных данных, экспериментальное значение сопоставлялось не с точной формулой, а с верхней оценкой через сумму логарифмов высот.

3.1.2 Параметры и методика эксперимента

Эксперимент выполнялся для размеров входных данных $n = 2^k, k = 1, 2, \dots, 14$, то есть для $n \in \{2, 4, 8, 16, \dots, 16384\}$. Выбор степеней двойки обусловлен двумя причинами. Во-первых, сортировка Бэтчера и битоническая сортировка в классическом виде определены именно для размеров, являющихся степенями двойки. Во-вторых, такой выбор позволяет непосредственно сопоставлять экспериментальные данные с формулами, выведенными во второй главе через замену $n = 2m$. Для каждого значения n формировались три вида входных данных:

1. `data_random` — случайная перестановка чисел от 0 до $n-1$;
2. `data_sorted` — уже отсортированный массив;
3. `data_reverse` — обратно отсортированный массив

Случайные данные использовались для сортировки Бэтчера, битонической сортировки, сортировки слиянием, пирамидальной сортировки и Quick Sort в среднем случае. Отсортированные данные использовались для худшего случая Quick Sort при выборе первого элемента в качестве опорного. Обратно отсортированные данные использовались для худшего случая сортировки вставками. Для воспроизводимости эксперимента генераторы случайных чисел были зафиксированы. Для случайной стратегии Quick Sort число сравнений усреднялось по нескольким независимым запускам. В ноутбуке использовалось значение $NUM_TRIALS = 5$. Теоретические значения вычислялись отдельными функциями. Например, для сортировки Бэтчера использовалась формула

```
1 def theory_batcher_sort(n):
2     m = math.log2(n)
3     return n * m * (m - 1) / 4 + n - 1
```

Рисунок 3.2 Вычисление теоретического числа компараторов сортировки Бэтчера

Аналогично были заданы функции для битонической сортировки, Merge Sort, Quick Sort, Insertion Sort, Heap Sort и Stooge Sort. Для визуализации результатов использовались библиотеки `numpy` и `matplotlib`. Графики строились в двух шкалах: обычной линейной и логарифмической по обеим осям. Логарифмическая шкала особенно удобна для сравнения функций роста различных порядков, поскольку на ней степенные и логарифмические различия становятся более наглядными.

3.2 Сопоставление теоретических и эмпирических результатов

В данном разделе приводится сопоставление фактического числа сравнений, полученного в ходе выполнения программ, с теоретическими формулами, выведенными во второй главе.

3.2.1 Верификация формул для сортировки Бэтчера

Для сортировки Бэтчера теоретическое число компараторов полной сортировочной сети определяется формулой

$$S(n) = \frac{n \log^2 n (\log^2 n - 1)}{4} + n - 1. \quad (3.1)$$

Так как сортировка Бэтчера является сортировочной сетью, число сравнений не зависит от исходного порядка элементов. Поэтому эмпирическое значение должно совпадать с (3.1) для любого входа длины n . Таблица 3.1 демонстрирует значения, получаемые по формуле (3.1), для размеров, использованных в эксперименте.

Таблица 3.1 — Теоретическое число компараторов сортировки Бэтчера

n	$\log_2 n$	$S(n)$
2	1	1
4	2	5
8	3	19
16	4	63
32	5	191
64	6	543
128	7	1471
256	8	3839
512	9	9727
1024	10	24063
2048	11	58367
4096	12	139263
8192	13	327679
16384	14	761855

В экспериментальном ноутбуке для каждого из этих значений n выполняется запуск функции `batcher_sort(data_random)`, после чего фактическое число сравнений сравнивается со значением функции `theory_batcher_sort(n)`. Для всех размеров входа отклонение равно нулю: $\Delta n = S_{\{emp\}}(n) - S_{\{theory\}}(n) = 0$.

Это подтверждает корректность как программной реализации сортировки Бэтчера, так и точной формулы (3.1).

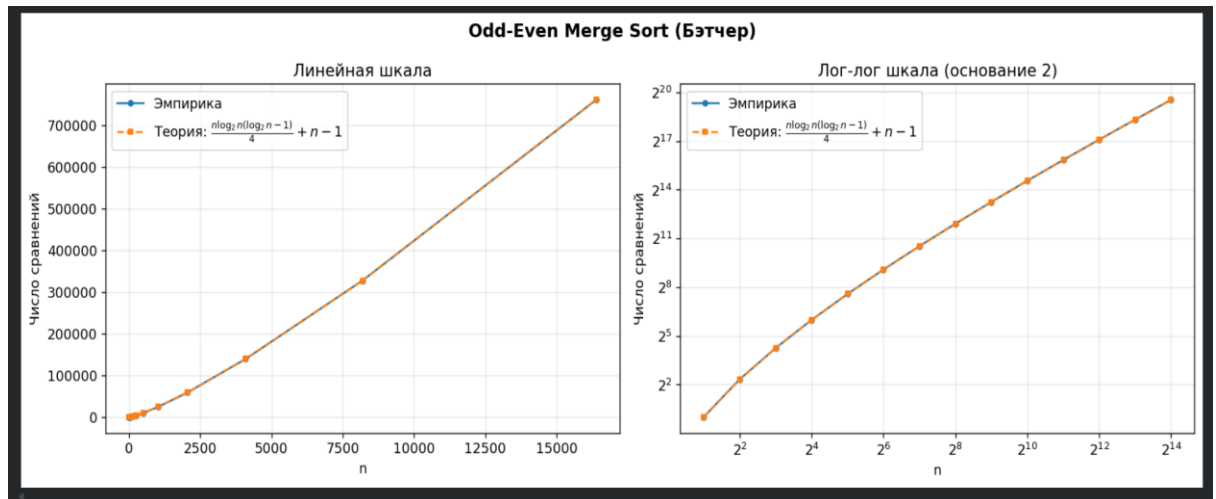


Рисунок 3.3 - Сопоставление эмпирического и теоретического числа сравнений для сортировки Бэтчера.

Для битонической сортировки аналогичная формула имеет вид.

$$S_B(n) = \frac{n \log_2 n (\log_2 n + 1)}{4}. \quad (3.2)$$

Поскольку битоническая сортировка также является сортировочной сетью, отклонение между эмпирическим и теоретическим числом сравнений также равно нулю. Это дополнительно подтверждает корректность подхода к программной реализации сортировочных сетей со счётчиками сравнений.

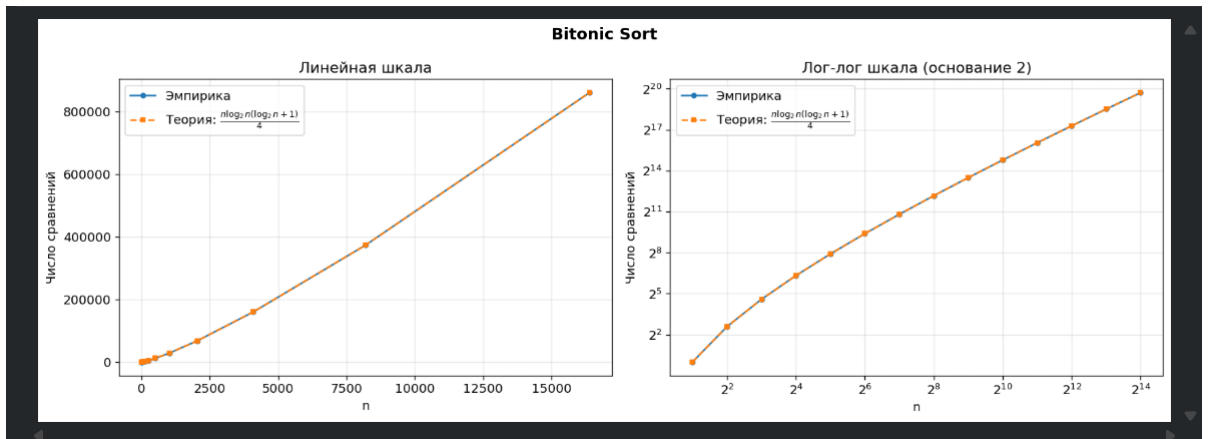


Рисунок 3.4 - Сопоставление эмпирического и теоретического числа сравнений для битонической сортировки.

3.2.2 Верификация формул для классических сортировок

Для сортировки слиянием в худшем случае при $n = 2^m$ используется оценка

$$T_{\{merge\}}(n) = n \log^2 n - n + 1. \quad (3.3)$$

На случайных данных фактическое число сравнений может быть немного ниже худшего случая, так как при слиянии одна из половин может исчерпаться раньше. Поэтому для Merge Sort ожидалось не строгое равенство, а близость эмпирической кривой к теоретической верхней оценке. Эксперимент подтвердил это поведение: график Merge Sort располагается ниже или вблизи кривой (3.3), сохраняя тот же порядок роста $\theta(n \log n)$

Для Quick Sort в худшем случае при выборе первого элемента в качестве опорного на отсортированном входе теоретическая сложность имеет квадратичный порядок. В зависимости от конкретной реализации разбиения точная формула для числа сравнений может отличаться на линейное слагаемое. В ноутбуке использовалась оценка

$$T_{\{quick, worst\}}(n) = \frac{n(n+1)}{2} - 1. \quad (3.4)$$

Для среднего случая Quick Sort использовалась формула

$$T_{\{quick,avg\}}(n) = 2(n + 1)H_n - 4n, \quad (3.5)$$

где H_n - n-е гармоническое число. Поскольку фактический результат зависит от случайного выбора опорных элементов, экспериментальное значение усреднялось по нескольким запускам. Полученная эмпирическая кривая хорошо согласуется с теоретической оценкой $\Theta(n \log n)$, хотя для малых n возможны заметные колебания.

Для сортировки вставками в худшем случае на обратно отсортированном массиве точная формула имеет вид

$$T_{\{insertion\}}(n) = \frac{n(n-1)}{2}. \quad (3.6)$$

Экспериментальное число сравнений совпадает с этой формулой, поскольку каждый новый элемент в обратно отсортированном массиве сравнивается со всеми предыдущими.

Для пирамидальной сортировки проверялась не точная формула, а верхняя оценка через суммарную стоимость операций sift_down. Экспериментальное число сравнений оказалось ниже верхней оценки, но сохраняет тот же порядок роста:

$$T_{\{heap\}}(n) \in \Theta(n \log n). \quad (3.7)$$

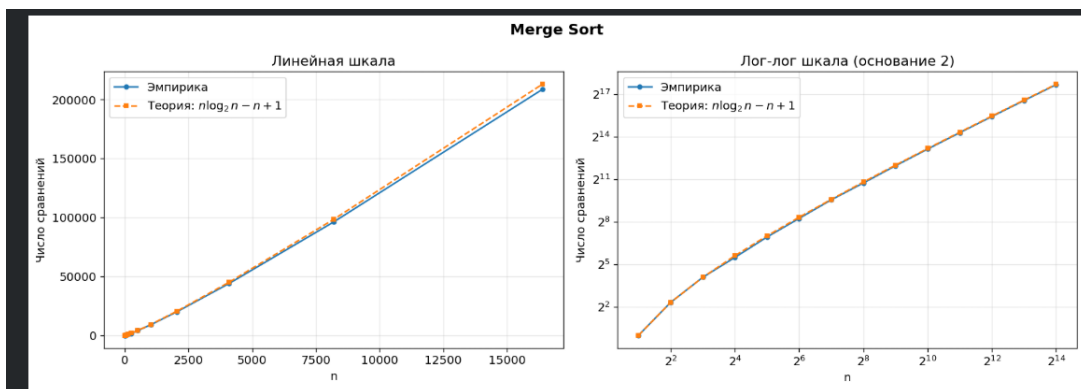


Рисунок 3.5 - Сопоставление эмпирики и теории для MergeSort

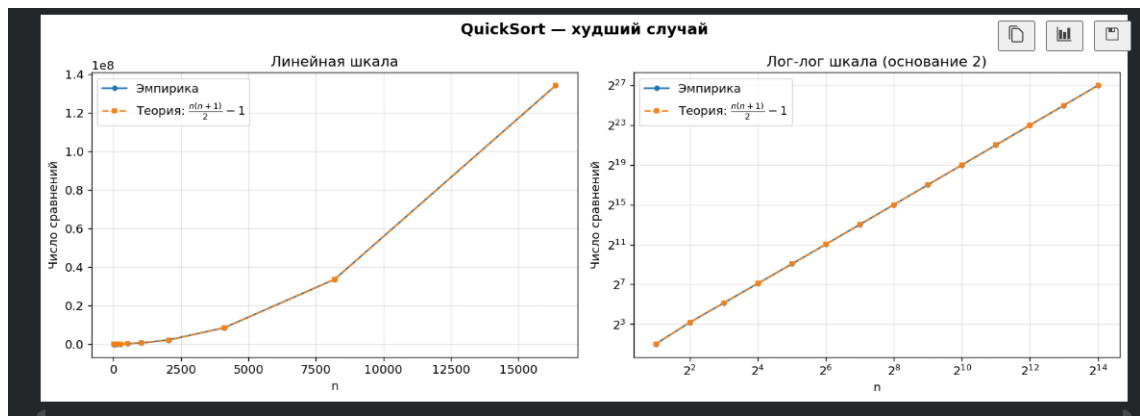


Рисунок 3.6 - Сопоставление эмпирики и теории для QuickSort

3.2.3 Сводный сравнительный анализ алгоритмов

Для общего сравнения алгоритмов был построен сводный график эмпирического числа сравнений в логарифмических координатах. На нём одновременно отображены:

1. Сортировка Бэтчера;
2. Битоническая сортировка;
3. Сортировка слиянием;
4. Quick Sort в среднем случае;
5. Heap Sort;
6. Сортировка вставками в худшем случае;
7. Quick Sort в худшем случае;
8. Stooge Sort для малых размеров входа.

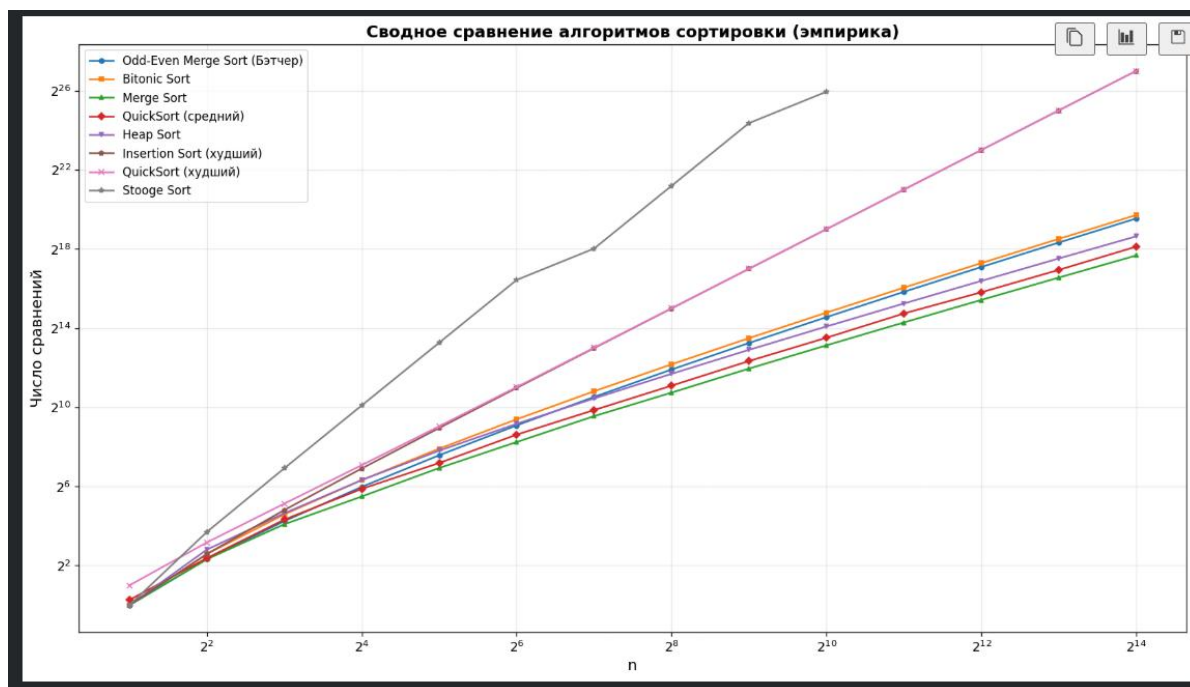


Рисунок 3.7 - Сводное сравнение алгоритмов сортировки по числу сравнений

Логарифмический график позволяет сделать несколько выводов.

Во-первых, сортировка слиянием, Quick Sort в среднем случае и Heap Sort располагаются в области функций порядка $\Theta(n \log n)$. Их графики растут существенно медленнее, чем графики квадратичных алгоритмов.

Во-вторых, сортировка Бэтчера и битоническая сортировка имеют порядок $\Theta(n \log 2n)$. На практике их число сравнений больше, чем у Merge Sort и Heap Sort, однако они обладают важным преимуществом: фиксированной структурой и высокой параллельностью.

В-третьих, сортировка вставками и Quick Sort в худшем случае демонстрируют квадратичный рост. При увеличении n их графики быстро уходят выше графиков алгоритмов класса $\Theta(n \log n)$ и $\Theta(n \log 2n)$.

В-четвёртых, Stooge Sort имеет ещё более быстрый рост порядка $\Theta(n^{2.709})$ и становится практически непригодным уже при относительно небольших размерах входа. Поэтому в эксперименте он ограничен значениями $n \leq 1024$.

Сводные результаты сопоставления представлены в таблице 3.2.

Таблица 3.2 — Экспериментально подтверждённые классы сложности

Алгоритм	Условия эксперимента	Теоретическая оценка	Характер совпадения
Сортировка Бэтчера	случайный вход, $n=2k$	$\theta(n \log 2n)$	точное совпадение с формулой
Bitonic Sort	случайный вход, $n=2k$	$\theta(n \log 2n)$	точное совпадение с формулой
Merge Sort	случайный вход	$\theta(n \log n)$	близость к верхней оценке
Quick Sort, средний случай	случайный pivot, усреднение	$\theta(n \log n)$	статистическое согласование
Quick Sort, худший случай	отсортированный вход, первый pivot	$\theta(n^2)$	совпадение с реализационной формулой
Insertion Sort, худший случай	обратно отсортированный вход	$\theta(n^2)$	точное совпадение
Heap Sort	случайный вход	$\theta(n \log n)$	ниже верхней оценки, тот же порядок
Stooge Sort	случайный вход, $n \leq 1024$	$\theta(n^{2.709})$	совпадение с рекуррентной формулой

Для сортировочных сетей совпадение является точным, поскольку число компараторов не зависит от входа. Для алгоритмов, зависящих от данных, проверяется не столько точная равенство с формулой, сколько соответствие эмпирической кривой ожидаемому порядку роста.

3.3 Практическая значимость результатов

Экспериментальные результаты позволяют перейти от формальных асимптотических оценок к практическим выводам о применимости рассмотренных алгоритмов. При выборе алгоритма сортировки важны не только класс сложности, но и устойчивость, требования к памяти, предсказуемость времени работы, пригодность к параллельному исполнению и зависимость от входных данных.

3.3.1 Рекомендации по выбору алгоритма сортировки

На основании теоретического и экспериментального анализа можно сформулировать следующие рекомендации.

Для универсальной последовательной сортировки больших массивов предпочтительны алгоритмы класса $\Theta(n \log n)$: сортировка слиянием, пирамидальная сортировка и современные гибридные алгоритмы. Merge Sort удобен, когда требуется устойчивость, но требует дополнительной памяти $\Theta(n)$. Heap Sort работает на месте и имеет гарантированный худший случай $\Theta(n \log n)$, но на практике часто уступает более кэш-дружественным алгоритмам.

Quick Sort эффективен на практике и имеет среднюю сложность $\Theta(n \log n)$, однако его худший случай равен $\Theta(n^2)$ [2, 15]. Поэтому при использовании Quick Sort необходимо применять устойчивые к худшим случаям стратегии выбора опорного элемента: случайный pivot, медиана трёх, introsort.

Сортировка вставками не подходит для больших массивов, поскольку в худшем случае имеет сложность $\Theta(n^2)$. Однако она эффективна на малых и почти отсортированных входах и поэтому часто применяется как вспомогательный алгоритм внутри гибридных сортировок.

Сортировка Бэтчера не является оптимальной по числу сравнений при последовательном исполнении: её сложность $\Theta(n \log 2n)$ выше нижней границы $\Omega(n \log n)$. Однако её преимущества проявляются в других условиях:

1. фиксированное число сравнений;
2. отсутствие зависимости от входных данных;
3. высокая степень параллелизма;
4. регулярная структура;
5. удобство аппаратной реализации.

Поэтому сортировка Бэтчера целесообразна в задачах, где важны не минимальное число сравнений в последовательной модели, а предсказуемость и параллельность выполнения. Битоническая сортировка имеет тот же асимптотический порядок, что и сортировка Бэтчера, и также является сортировочной сетью [3, 18]. Она широко используется в GPU-сортировках

благодаря простой регулярной структуре. При этом точное число компараторов у неё отличается от сортировки Бэтчера. Таблица 3.3 содержит обобщённые рекомендации.

Таблица 3.3 — Рекомендации по выбору алгоритма сортировки

Условия задачи	Рекомендуемый алгоритм	Обоснование
Большой массив, нужна устойчивость	Merge Sort	$\Theta(n \log n)$, устойчивость
Большой массив, ограниченная память	Heap Sort	$O(1)$ доп. памяти, худший случай $\Theta(n \log n)$
Практическая сортировка общего назначения	Quick Sort	высокая средняя эффективность
Малые или почти отсортированные массивы	Insertion Sort	низкие накладные расходы
Параллельная аппаратная реализация	Бэтчер / Bitonic Sort	регулярная сеть, глубина $\Theta(\log^2 n)$
Постоянное время исполнения независимо от входа	Бэтчер / Bitonic Sort	отсутствие зависимостей от данных
Учебная демонстрация границ метода	Stooge Sort, Shellsort,	разные типы рекуррентностей и методов анализа

3.3.2. Применимость результатов в параллельных и криптографических системах

Практическая значимость сортировки Бэтчера связана прежде всего с её сетевой природой. В отличие от Quick Sort или Heap Sort, сортировочная сеть заранее определяет все пары сравниваемых индексов. Это свойство важно в трёх классах задач. Следовательно, при наличии достаточного числа параллельных компараторов сортировка может выполняться за полилогарифмическое число тактов. Средний параллелизм сети оценивается как $P(n) = \frac{S(n)}{d_{S(n)}} \approx \frac{n}{2}$. В аппаратных схемах важно, чтобы структура вычислений была регулярной и заранее известной. Сортировка Бэтчера удовлетворяет этому требованию: компараторы и связи между ними определяются только n . Поэтому сеть может быть синтезирована как фиксированная схема без условных переходов и ветвлений. В криптографии

существенным требованием является защита от побочных каналов, в частности от утечек через время исполнения. Алгоритмы, структура которых зависит от данных, могут раскрывать информацию о входе через различия во времени работы или шаблоне обращений к памяти. Сортировочные сети, напротив, выполняют одну и ту же последовательность операций для любых входных данных, что делает их полезными при построении constant-time алгоритмов.

Кроме того, результаты раздела 2.4 показывают методологическую связь сортировочных сетей с криптографическими алгоритмами, основанными на рекурсивных преобразованиях. Особенно важна аналогия между чётно-нечётным слиянием Бэтчера и алгоритмом Кули–Тьюки FFT/NTT: оба алгоритма используют разбиение данных на чётные и нечётные индексы и имеют рекуррентность с двумя подзадачами половинного размера. Поэтому единый метод анализа рекуррентностей применим как к сортировочным сетям, так и к преобразованиям, используемым в постквантовой криптографии [25].

Выводы по третьей главе

В третьей главе была проведена экспериментальная проверка теоретических результатов, полученных во второй главе. На основании машинного эксперимента можно сформулировать следующие выводы.

1. Разработаны программные реализации основных рассматриваемых алгоритмов сортировки на языке Python со встроенными счётчиками сравнений. Реализации позволяют напрямую измерять фактическое число сравнений, выполняемых алгоритмом на входах заданного размера.

2. Для сортировки Бэтчера экспериментально подтверждена точная формула

$$S(n) = \frac{n \log^2 n (\log^2 n - 1)}{4} + n - 1.$$

Во всех проведённых экспериментах эмпирическое число сравнений совпадает с теоретическим значением.

3. Для битонической сортировки также получено точное совпадение эмпирических результатов с теоретической формулой. Это подтверждает корректность реализации сортировочных сетей со счётчиками компараторов.
4. Для классических алгоритмов сортировки эксперимент показал согласование с известными асимптотическими оценками: Merge Sort, Heap Sort и Quick Sort в среднем случае демонстрируют рост порядка $\Theta(n \log n)$ сортировка вставками и Quick Sort в худшем случае — рост порядка $\Theta(n^2)$
5. Сводный график в логарифмических координатах наглядно подтверждает различие между классами сложности.
6. Практический анализ показал, что сортировка Бэтчера уступает оптимальным последовательным алгоритмам по числу сравнений, но обладает преимуществами, важными для параллельных и криптографических систем: фиксированная структура, отсутствие зависимости от входных данных, высокая степень параллелизма и пригодность к аппаратной реализации.
7. Полученные экспериментальные результаты подтверждают корректность теоретических формул и служат основанием для практических рекомендаций по выбору алгоритма сортировки в зависимости от условий задачи.

ЗАКЛЮЧЕНИЕ

В настоящей выпускной квалификационной работе был проведён теоретический и экспериментальный анализ сложности алгоритмов сортировки методом рекуррентных соотношений. Центральным объектом исследования выступила чётно-нечётная сортировка слиянием Бэтчера — сортировочная сеть, обладающая фиксированной структурой сравнений, не зависящей от входных данных, и высокой степенью параллелизма [3, 18].

Цель работы заключалась в разработке и систематическом применении единого метода асимптотического анализа, основанного на решении рекуррентных соотношений, к сортировке Бэтчера и родственным ей алгоритмам, получении точных замкнутых формул сложности и экспериментальной проверке полученных результатов. В ходе выполнения работы поставленная цель была достигнута, а сформулированные во введении задачи решены в полном объёме.

В первой главе были рассмотрены теоретические основы асимптотического анализа алгоритмов сортировки. Была дана классификация алгоритмов сортировки по ключевым признакам: природе операций над элементами, асимптотической сложности, использованию дополнительной памяти, устойчивости, зависимости от входных данных и пригодности к параллельному исполнению. Особое внимание было уделено сортировочным сетям как классу алгоритмов, в которых последовательность компараторов фиксирована заранее и определяется только размером входа. Было показано, что основными характеристиками сортировочной сети являются её размер, то есть число компараторов, и глубина, то есть длина критического пути зависимостей.

Также в первой главе были систематизированы основные методы асимптотического анализа: асимптотические обозначения; метод раскрутки рекуррентных соотношений; мастер-теорема; метод Акра–Баззи; метод подстановки; метод дерева рекурсии; прямой подсчёт; вероятностный и

амортизированный анализ. Было показано, что метод рекуррентных соотношений является основным инструментом анализа алгоритмов с рекурсивной структурой, а для алгоритмов, не укладывающихся в парадигму «разделяй и властвуй», требуются альтернативные методы.

Во второй главе был выполнен основной математический анализ. Для операции чётно-нечётного слияния Бэтчера было записано и решено рекуррентное соотношение: $M(n) = 2M(\frac{n}{2}) + \frac{n}{2} - 1$, $M(2) = 1$. Методом раскрутки была получена точная формула $M(n) = \frac{n}{2}(\log_2 n - 1) + 1$, из которой следует асимптотическая оценка $M(n) \in \Theta(n \log n)$.

Для полной сортировочной сети Бэтчера было получено рекуррентное соотношение: $S(n) = 2S(\frac{n}{2}) + M(n)$, решение которого дало точную формулу числа компараторов: $S(n) = \frac{n \log^2 n (\log^2 n - 1)}{4} + n - 1$. Следовательно, размер сети Бэтчера имеет асимптотику: $S(n) \in \Theta(n \log^2 n)$. Отдельно была рассмотрена глубина сортировочной сети как характеристика параллельной сложности. Было показано, что глубина операции слияния равна $\log_2 n$, а глубина полной сортировки определяется формулой: $d_{S(n)} = \frac{\log^2 n (\log^2 n + 1)}{2}$. Таким образом, параллельная сложность сортировки Бэтчера составляет $\Theta(\log^2 n)$, что существенно меньше её последовательной сложности по числу компараторов. Этот результат объясняет практическую значимость сортировки Бэтчера для аппаратных и параллельных реализаций.

В этой же главе метод рекуррентных соотношений был применён к классическим алгоритмам сортировки. Было показано, что сортировка слиянием имеет сложность $\Theta(n \log n)$, битоническая сортировка — $\Theta(n \log 2n)$, быстрая сортировка в лучшем и среднем случаях — $\Theta(n \log n)$, а в худшем случае — $\Theta(n^2)$. Для сортировки вставками была получена квадратичная оценка $\Theta(n^2)$, а для пирамидальной сортировки — оценка $\Theta(n \log n)$ методом прямого суммирования.

На основании проведённого анализа были сформулированы границы применимости метода рекуррентных соотношений. Было установлено, что для прямого применения метода необходимы три условия: наличие рекурсивной структуры, детерминированность размеров подзадач и явная выразимость стоимости работы вне рекурсивных вызовов. Если хотя бы одно из этих условий нарушается, стандартная раскрутка рекуррентности либо требует существенной модификации, либо становится неприменимой. В качестве примеров были рассмотрены Quick Sort в среднем случае, Heap Sort, Shellsort и Tim Sort. Дополнительно во второй главе было показано, что метод рекуррентных соотношений применим не только к сортировкам, но и к алгоритмам, используемым в криптографических системах и вычислительной алгебре. Были рассмотрены умножение Карацубы, алгоритм Тоома–Кука, быстрое преобразование Фурье и теоретико-числовое преобразование, алгоритм Штрассена для матричного умножения и дерева Меркля. Это позволило продемонстрировать универсальность рассматриваемого метода для широкого класса алгоритмов парадигмы «разделяй и властвуй». Отдельно была выявлена структурная аналогия между чётно-нечётным слиянием Бэтчера и алгоритмом Кули–Тьюки FFT/NTT, поскольку оба алгоритма используют разбиение данных по чётности индексов и приводят к рекуррентностям с двумя подзадачами половинного размера.

В третьей главе была выполнена экспериментальная проверка теоретических результатов. Для этого были разработаны программные реализации рассматриваемых алгоритмов сортировки на языке Python со встроенными счётчиками сравнений. Эксперименты проводились для входов размера $n=2^k$, что соответствует рекурсивной структуре сортировки Бэтчера и битонической сортировки. Для сортировки Бэтчера было экспериментально подтверждено точное совпадение эмпирического числа сравнений с формулой

Таким образом, в работе были получены следующие основные результаты:

1. Систематизированы методы асимптотического анализа алгоритмов сортировки и показана область применимости каждого из них.
2. Проведён полный математический анализ сортировки Бэтчера методом раскрутки рекуррентных соотношений.
3. Получены точные замкнутые формулы для числа компараторов операции слияния, числа компараторов полной сортировки и глубины сети Бэтчера.
4. Выполнено сравнение сортировки Бэтчера с классическими алгоритмами сортировки и показано, какие алгоритмы допускают прямой анализ методом рекуррентных соотношений, а какие требуют альтернативных методов.
5. Метод рекуррентных соотношений распространён на ряд криптографических алгоритмов, имеющих рекурсивную структуру.
6. Проведена экспериментальная верификация полученных формул с использованием программных реализаций алгоритмов со счётчиками сравнений.

Теоретическая значимость работы состоит в систематизации методологии асимптотического анализа сортировочных сетей и алгоритмов парадигмы «разделяй и властвуй». Практическая значимость заключается в возможности использования полученных результатов при выборе алгоритмов сортировки для последовательных, параллельных и аппаратных систем, а также при изучении связи сортировочных сетей с вычислительными структурами, применяемыми в криптографических алгоритмах. Перспективами дальнейшего исследования являются расширение экспериментальной части за счёт анализа реального времени выполнения на различных аппаратных платформах, исследование реализации сортировочных сетей на GPU и FPGA, а также более детальное сопоставление сортировочных сетей с вычислительными сетями FFT/NTT, используемыми в пост квантовой криптографии.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Кнут, Д. Э. Искусство программирования. Том 1. Основные алгоритмы / Д. Э. Кнут. — 3-е изд. — Москва: Вильямс, 2022. — 720 с.
2. Кормен, Т. Алгоритмы: построение и анализ / Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн. — 3-е изд. — Москва: Вильямс, 2022. — 1328 с.
3. Batcher, K. E. Sorting Networks and their Applications / K. E. Batcher // Proceedings of the AFIPS Spring Joint Computer Conference. — 1968. — Vol. 32. — P. 307–314.
4. Akra, M. On the solution of linear recurrence equations / M. Akra, L. Bazzi // Computational Optimization and Applications. — 1998. — Vol. 10, No. 2. — P. 195–210.
5. Geissmann, B. Quadratic Time Algorithms Appear to be Optimal for Sorting Evolving Data / B. Geissmann, S. Leucci, C.-H. Liu, P. Penna // Algorithmica. — 2020. — Vol. 82. — P. 2983–3011. — DOI: 10.1007/s00453-020-00719-z.
6. Braverman, M. Optimal Bounds for Noisy Sorting / M. Braverman, J. Mao, S. M. Weinberg // Proceedings of the 55th Annual ACM Symposium on Theory of Computing (STOC 2023). — New York: ACM, 2023. — P. 1502–1515. — DOI: 10.1145/3564246.3585184.
7. Cazenave, T. A Probabilistic Analysis of Optimization by Probabilistic Tree Search / T. Cazenave // Proceedings of the 2019 IEEE Conference on Games. — 2019. — P. 45–52.
8. Kim, H. An efficient indexing for deep learning-based visual place recognition using tree structures / H. Kim [et al.] // Scientific Reports. — 2023. — Vol. 13. — URL: <https://www.nature.com/articles/s41598-023-27986-6> (дата обращения: 26.04.2026).
9. Ахо, А. В. Структуры данных и алгоритмы / А. В. Ахо, Дж. Э. Хопкрофт, Дж. Д. Ульман. — Москва: Вильямс, 2016. — 400 с.

- 10.Клейнберг, Дж. Алгоритмы: разработка и применение / Дж. Клейнберг, Е. Тардос. — Москва: Вильямс, 2022. — 880 с.
- 11.Skiena, S. S. The Algorithm Design Manual / S. S. Skiena. — 3rd ed. — Cham: Springer, 2020. — 794 p.
- 12.Goodrich, M. T. Data Structures and Algorithms in Python / M. T. Goodrich, R. Tamassia, M. H. Goldwasser. — Hoboken: Wiley, 2019. — 770 p.
- 13.Окулов, С. М. Программирование в алгоритмах / С. М. Окулов. — Москва: БИНОМ. Лаборатория знаний, 2017. — 383 с.
- 14.Mehlhorn, K. Data Structures and Algorithms 1: Sorting and Searching / K. Mehlhorn. — Berlin: Springer, 2013. — 336 p.
- 15.Sedgewick, R. Algorithms in C, Parts 1–5: Fundamentals, Data Structures, Sorting, Searching, and Graph Algorithms / R. Sedgewick. — 3rd ed. — Boston: Addison-Wesley Professional, 2001. — 720 p.
- 16.Weiss, M. A. Data Structures and Algorithm Analysis in C++ / M. A. Weiss. — 4th ed. — Boston: Pearson, 2013. — 688 p.
- 17.Brass, P. Advanced Data Structures / P. Brass. — Cambridge: Cambridge University Press, 2008. — 474 p.
- 18.Кнут, Д. Э. Искусство программирования. Том 3. Сортировка и поиск / Д. Э. Кнут. — 2-е изд. — Москва: Вильямс, 2020. — 832 с.
- 19.Karatsuba, A. Multiplication of multidigit numbers on automata / A. Karatsuba, Yu. Ofman // Soviet Physics Doklady. — 1963. — Vol. 7. — P. 595–596.
- 20.Toom, A. L. The complexity of a scheme of functional elements realizing the multiplication of integers / A. L. Toom // Soviet Mathematics Doklady. — 1963. — Vol. 3. — P. 714–716.
- 21.Cooley, J. W. An algorithm for the machine calculation of complex Fourier series / J. W. Cooley, J. W. Tukey // Mathematics of Computation. — 1965. — Vol. 19, No. 90. — P. 297–301.
- 22.Strassen, V. Gaussian elimination is not optimal / V. Strassen // Numerische Mathematik. — 1969. — Vol. 13. — P. 354–356.

23. Schönhage, A. Schnelle Multiplikation großer Zahlen / A. Schönhage, V. Strassen // Computing. — 1971. — Vol. 7. — P. 281–292.
24. Merkle, R. C. A digital signature based on a conventional encryption function / R. C. Merkle // Advances in Cryptology — CRYPTO '87. — Berlin: Springer, 1988. — P. 369–378.
25. NIST. Post-Quantum Cryptography Standardization [Электронный ресурс]. — URL: <https://csrc.nist.gov/projects/post-quantum-cryptography> (дата обращения: 26.04.2026).
26. Pratt, V. R. Shellsort and Sorting Networks: PhD thesis / V. R. Pratt. — Stanford: Stanford University, 1971.
27. Ajtai, M. An $O(n \log n)$ sorting network / M. Ajtai, J. Komlós, E. Szemerédi // Proceedings of the 15th Annual ACM Symposium on Theory of Computing. — New York: ACM, 1983. — P. 1–9.
28. Auger, N. Merge strategies: from merge sort to TimSort / N. Auger, C. Nicaud, C. Pivoteau. — 2015. — URL: <https://hal.science/hal-01212839> (дата обращения: 26.04.2026).

ПРИЛОЖЕНИЯ

Приложение А. Исходный код экспериментальной верификации:

Приложение Б. Графики сравнения теоретических и эмпирических данных, не вошедшие в основной текст В

Приложение В. Исходный код верификации для необычных сортировок не вошедших в работу

В приложении А приведён исходный код основного машинного эксперимента, выполненного в среде Jupyter Notebook. В данном ноутбуке реализованы алгоритмы сортировки со встроенными счётчиками сравнений, функции для вычисления теоретических значений сложности, основной цикл эксперимента по размерам входа $n=2k$, а также построение графиков сопоставления теоретических и эмпирических результатов. Исходный код размещён в открытом репозитории проекта в файле:

https://github.com/mityamuch/Diplom_Batcher/blob/main/experiments/verification.ipynb

В состав приложения А входят следующие основные блоки:

1. Реализация чётно-нечётной сортировки слиянием Бэтчера.
2. Реализация битонической сортировки.
3. Реализация сортировки слиянием.
4. Реализация быстрой сортировки для среднего и худшего случаев.
5. Реализация сортировки вставками, пирамидальной сортировки и сортировки Студжа.
6. Функции вычисления теоретических значений сложности.
7. Основной цикл проведения эксперимента.
8. Построение графиков и таблиц отклонений.

В приложении Б приведены дополнительные графики, полученные при выполнении машинного эксперимента. В основной текст работы включены только наиболее показательные рисунки, необходимые для подтверждения результатов исследования. Остальные графики вынесены в приложение, чтобы не перегружать третью главу.

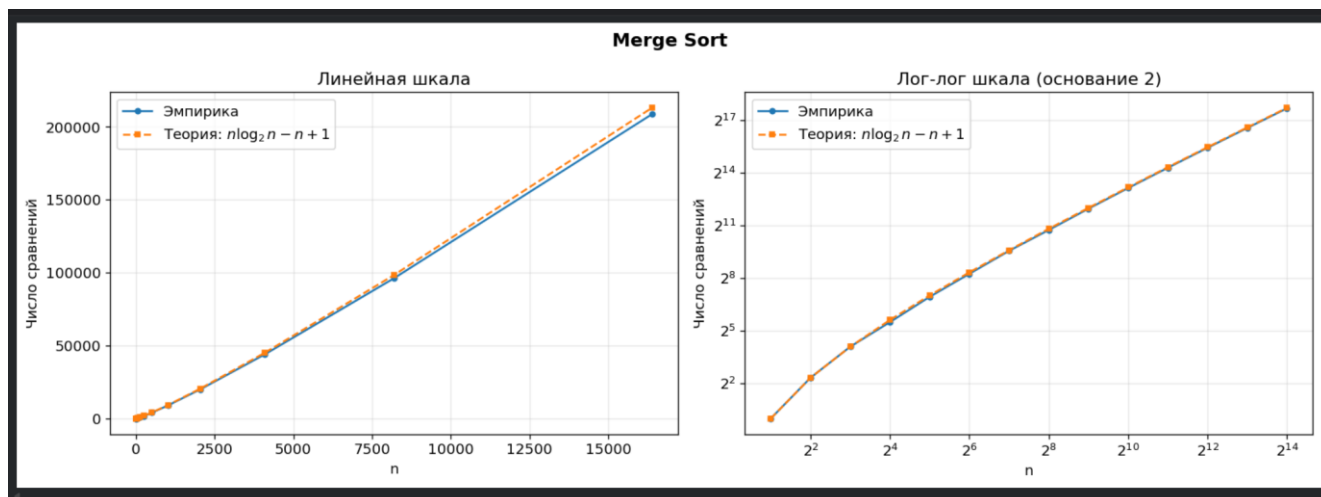


Рисунок Б.1 — Сопоставление эмпирических и теоретических данных для сортировки слиянием

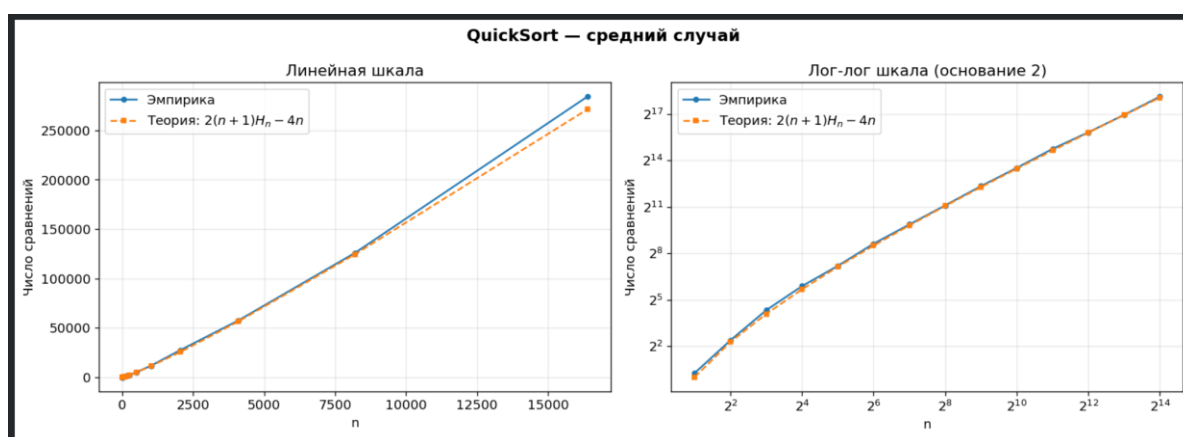


Рисунок Б.2 — Сопоставление эмпирических и теоретических данных для Quick Sort в среднем случае

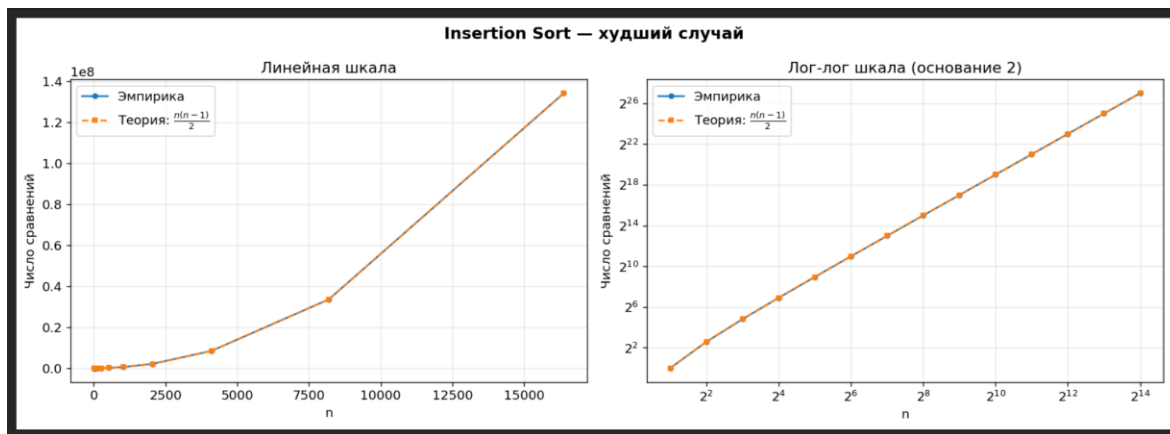


Рисунок Б.3 — Сопоставление эмпирических и теоретических данных для сортировки вставками

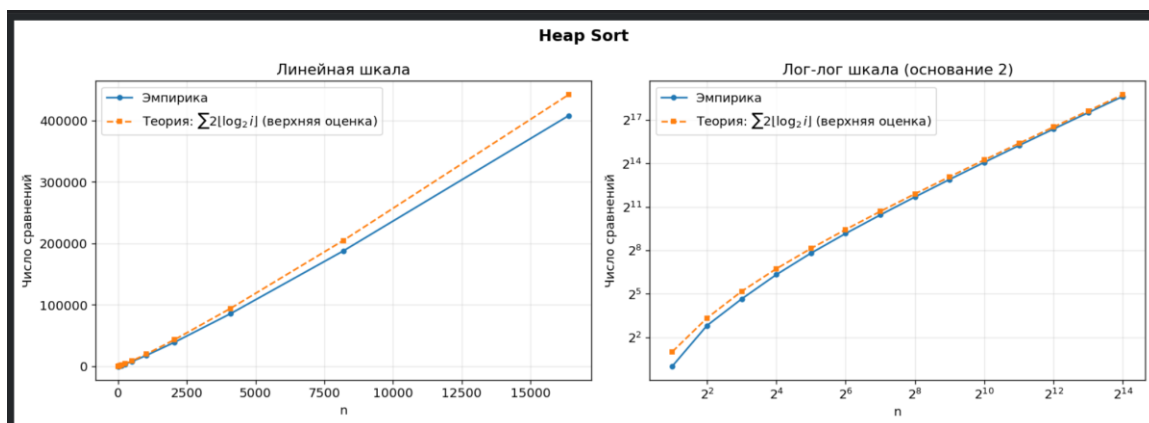


Рисунок Б.4 — Сопоставление эмпирических и теоретических данных для пирамидальной сортировки

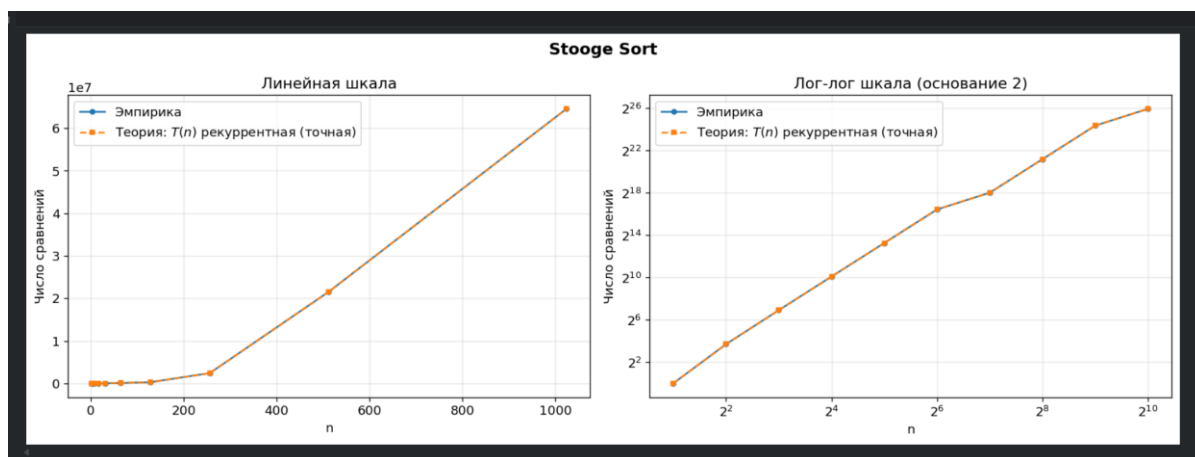


Рисунок Б.5 — Сопоставление эмпирических и теоретических данных для сортировки Студжа

В приложении В приведён исходный код дополнительного эксперимента с необычными алгоритмами сортировки, которые не вошли в основной текст выпускной квалификационной работы. Данный материал использовался как вспомогательный при изучении различных типов поведения алгоритмов сортировки и иллюстрирует различие между классическими и менее распространёнными подходами. Исходный код размещён в открытом репозитории проекта в файле:

https://github.com/mityamuch/Diplom_Batcher/blob/main/experiments/fun_sorts.ipynb

В ноутбуке представлены следующие алгоритмы:

1. Сортировка бармена (Cocktail Shaker Sort).
2. Сортировка гномов (Gnome Sort).
3. Сортировка расчёской (Comb Sort).

Для каждого алгоритма приведены программная реализация, краткое описание принципа работы, экспериментальный подсчёт числа сравнений и графики роста сложности на входах различных размеров.